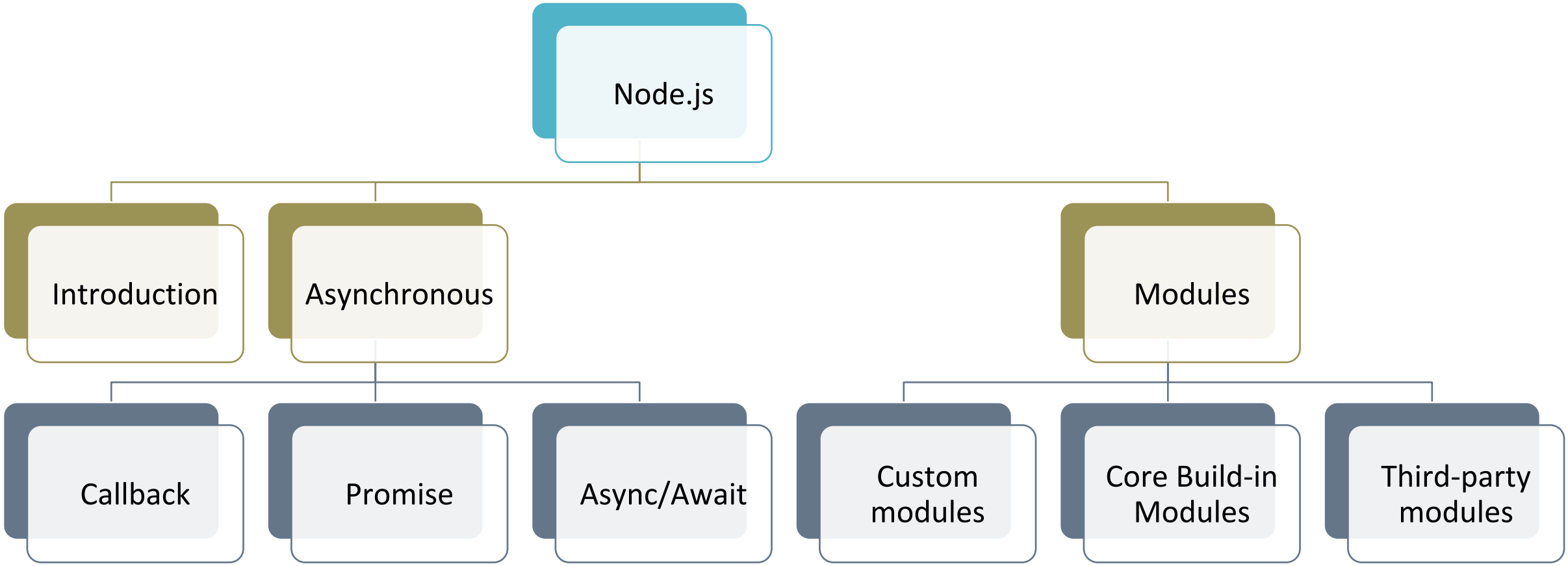
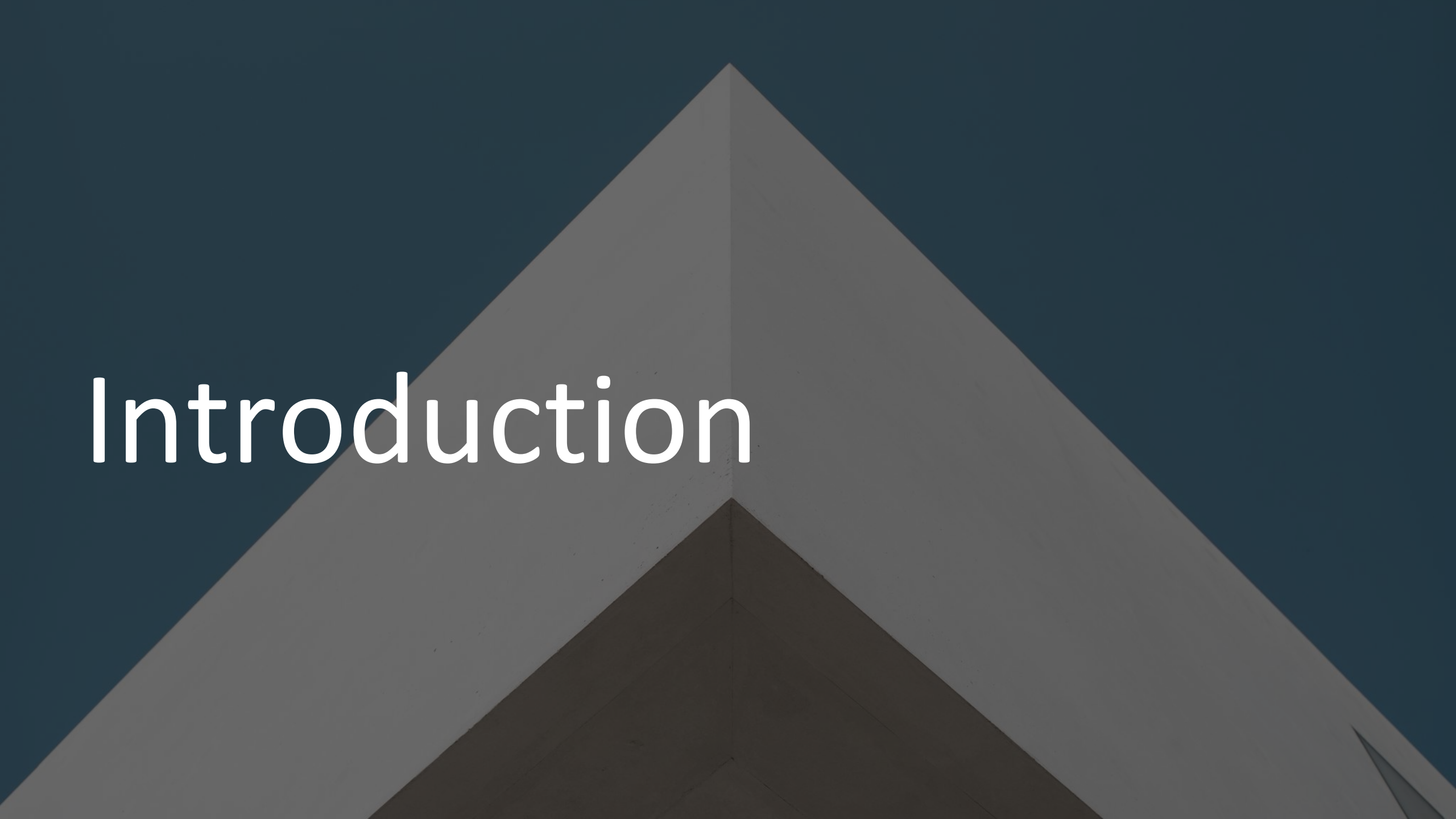


Node JS


Node.js
Introduction
Asynchronous
Modules
Callback
Promise
Async/Await
Custom modules
Core Build-in Modules
Third-party modules



Introduction

Node.js

Node.js is a free, open source cross-platform JavaScript **runtime environment** for executing **JavaScript** outside the browser, built on the V8 JavaScript engine.

Node.js excels at handling many simultaneous connections with minimal overhead, making it perfect for:

- Server-side web applications
- Real-time applications (chats, gaming, collaboration tools)
- APIs and microservices
- Data streaming applications
- Command-line tools





Browser



Node.js

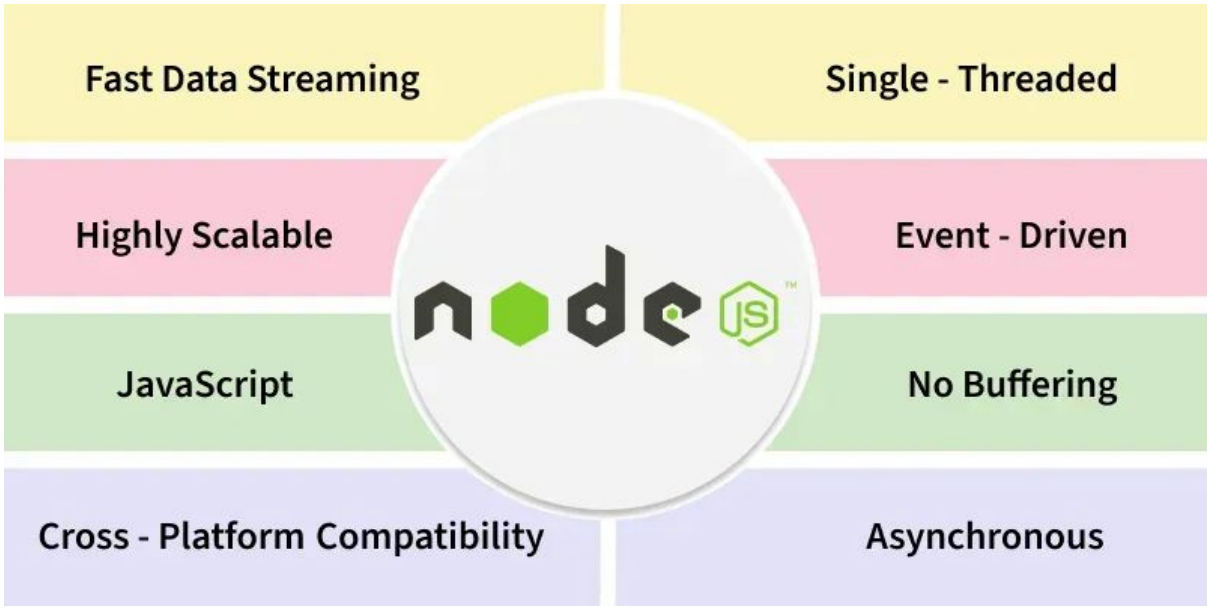


Node.js vs Browser

Feature	Node.js	Browser
File System Access	Yes	No
Networking (TCP/UDP)	Yes	No
DOM Access	No	Yes
Global Object	global	window/self
Modules	CommonJS/ESM	ESM/Scripts
Environment Variables	Yes (process.env)	No
Security	Full OS access	Sandboxed
Package Management	npm/yarn	CDN/Bundler

Node.js

It enables server-side development, supports asynchronous, event-driven programming, and efficiently handles scalable network applications



NodeJS is single-threaded, utilizing an event loop to handle multiple tasks concurrently.

It is asynchronous and non-blocking, meaning operations do not wait for execution to complete.

The V8 engine compiles JavaScript to machine code, making NodeJS fast and efficient.

Blocking I/O vs Non-Blocking I/O

BLOCKING I/O

OS needs to wait for a certain operation to complete before allowing the program to continue execution

user can't do anything else while waiting on a system call

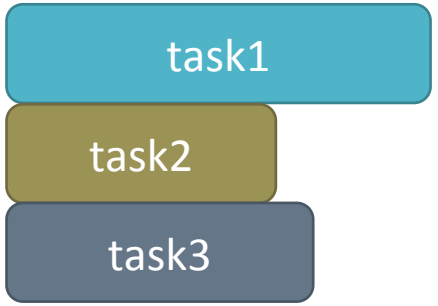


NON-BLOCKING I/O

known as asynchronous operations

It doesn't block the thread until they finish their work

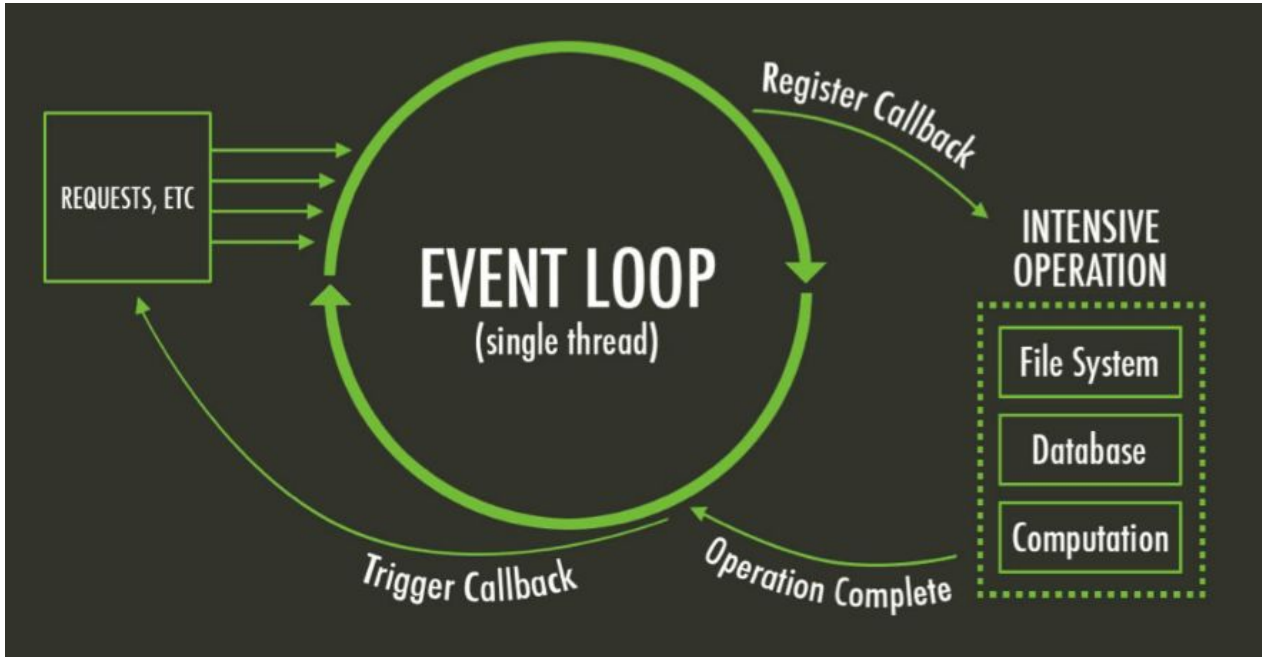
call returns immediately after completing its job without having any effect on what's happening in other threads



Node.js- architecture

Node.js employs a single-threaded model to handle many connections at once.

To achieve this, Node.js leverages an event-driven, non-blocking I/O model.



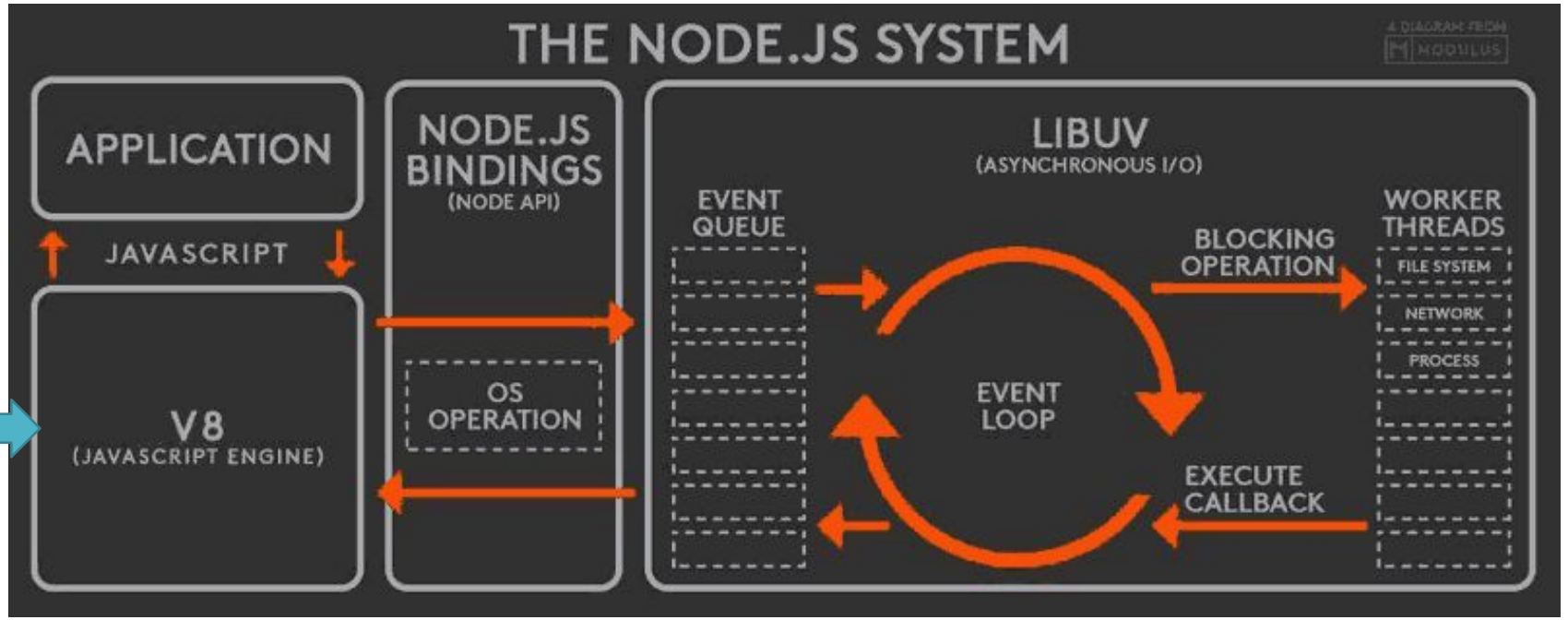

```
console.log("first line");

setTimeout(() => {
  console.log("timeout line");
  }, 2000);

console.log("last line");
```

```
first line
last line
timeout line
```

The **V8 engine** is Google's open-source JavaScript engine, used by Chrome and Node.js.
It compiles JavaScript to native machine code for fast execution.



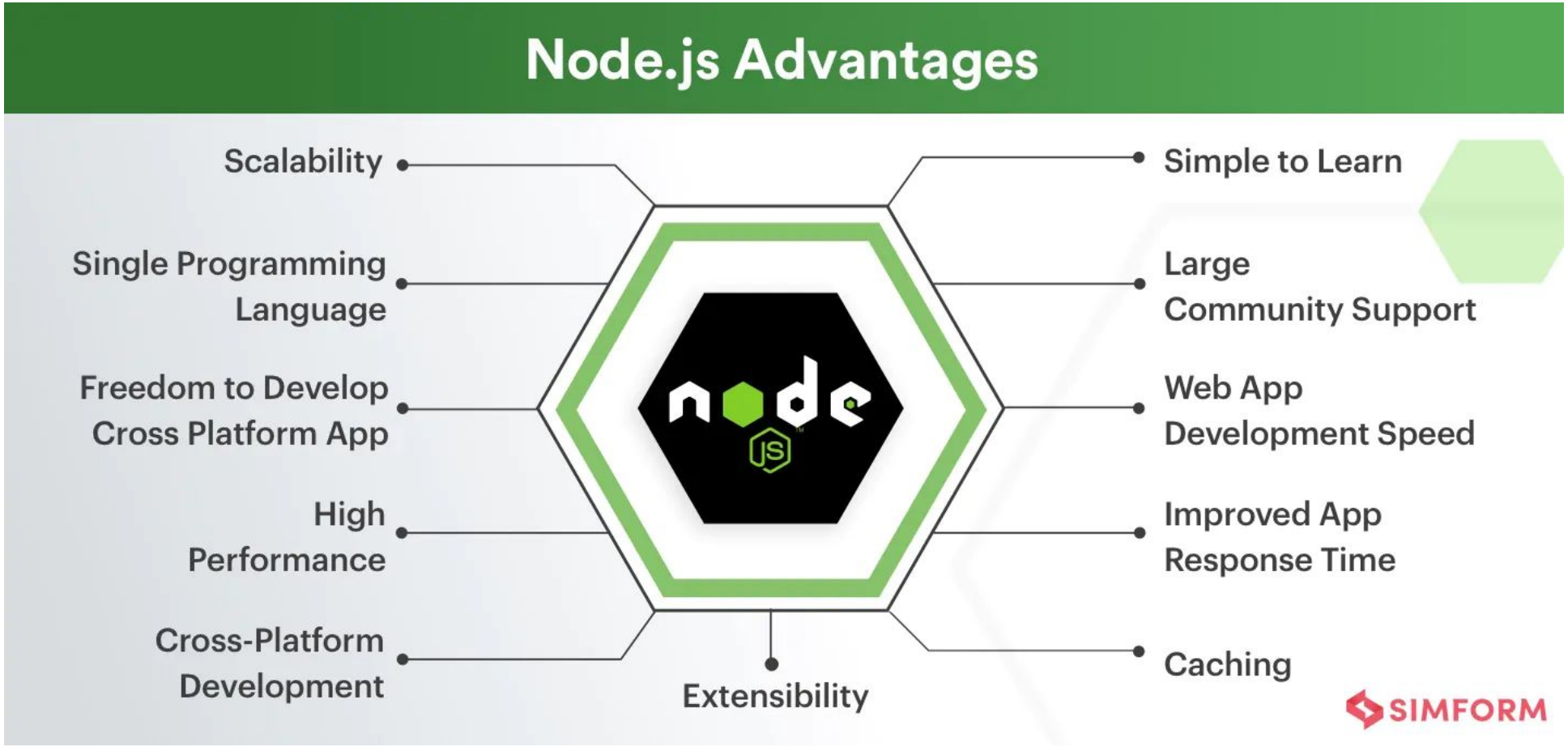
1. Client Request Phase
- Clients send requests to the Node.js server
- Each request is added to the **Event Queue**

2. Event Loop Phase
- The Event Loop continuously checks the **Event Queue**
- Picks up requests one by one in a loop

3. Request Processing
- Simple (non-blocking) tasks are handled immediately by the main thread
- Complex/blocking tasks are offloaded to the Thread Pool

4. Response Phase
- When blocking tasks complete, their callbacks are placed in the **Callback Queue**
- Event Loop processes callbacks and sends responses

https://medium.com/@golibraryco/is-nodejs-multi-threaded-how-does-nodejs-handle-multiple-requests-f77fb17bda49



https://www.simform.com/blog/nodejs-advantages-disadvantages/

Node.js Key Features

Server-Side JavaScript:

NodeJS allows JavaScript to run outside the browser, enabling backend development.

Asynchronous & Non-Blocking:

Uses an event-driven architecture to handle multiple requests without waiting, improving performance.

Single-Threaded Event Loop:

Efficiently manages concurrent tasks using a single thread, avoiding thread overhead.

Scalable & Lightweight:

single-threaded made capable of handling a huge number of simultaneous connections with high throughput

Speed

Non-blocking thread execution makes Node.js even faster and more efficient.

Node.js Key Features

V8 Chrome Engine:

High performance, Open-Source Javascript web assembly engine developed by google and rewoned for compiling JS into native machine code to execute it with exceptional speed..

Packages

A vast set of open-source Node.js packages is available

Strong backend

Node.js is written in C and C++, which makes it speedy and adds features like networking support.

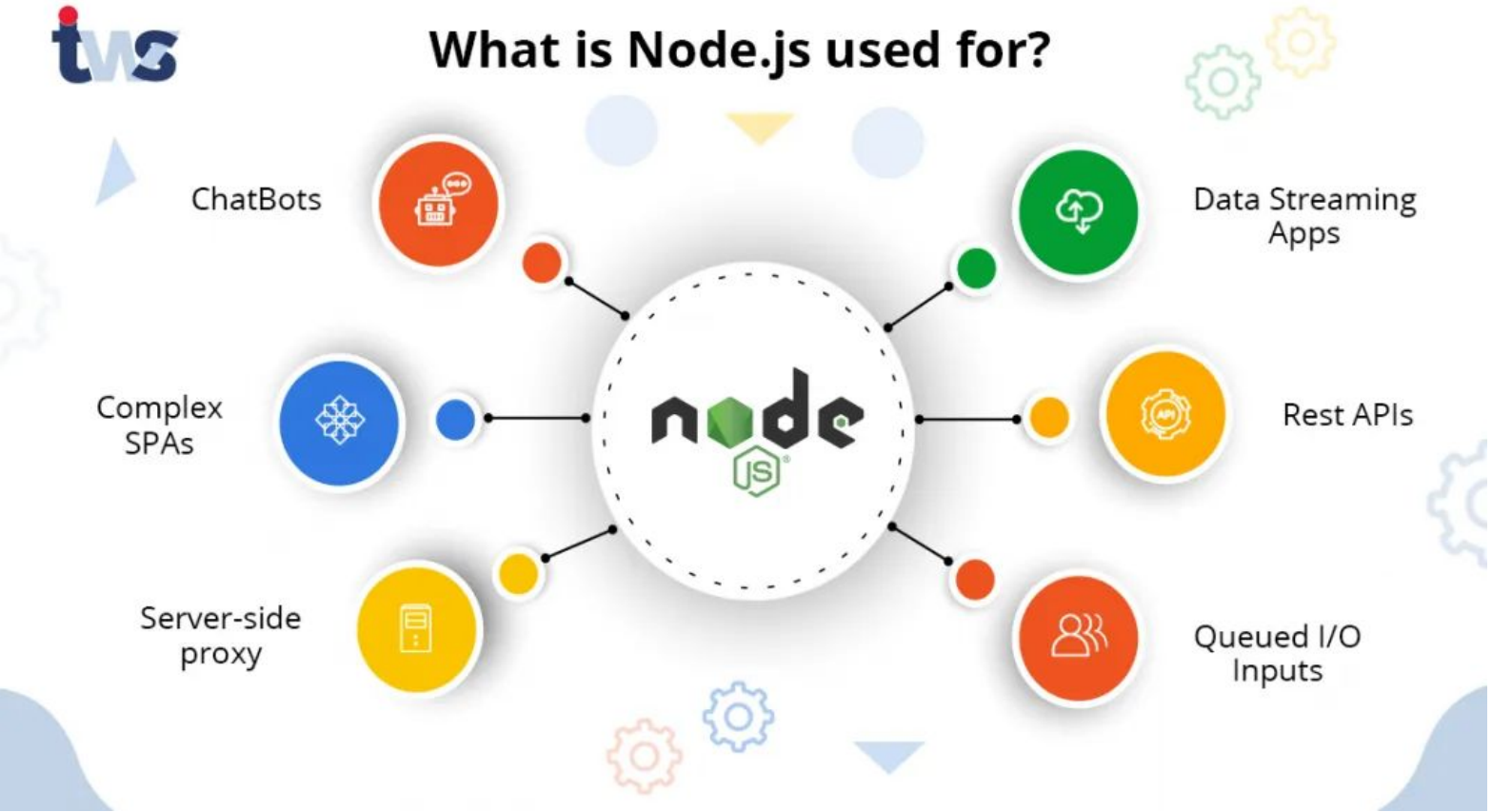
Multi-platform

Cross-platform support.

Maintainable

both the frontend and backend can be managed with JavaScript as a single language

Node.js Application

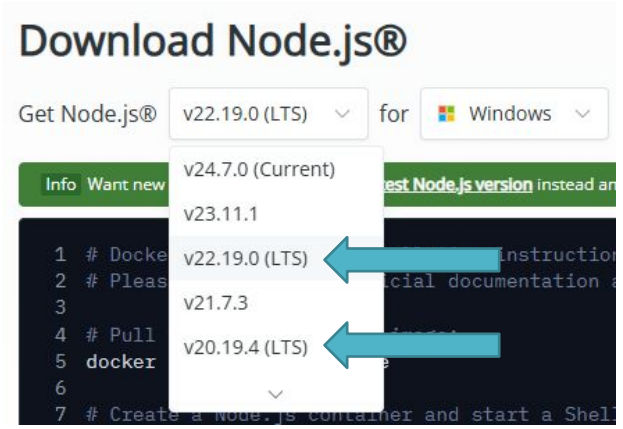


Single Page Application (SPA)

Get start

Download Node.js from official Node.js web site

https://nodejs.org/



Use an LTS (Long Term Support) version for production projects.

Install

Check version

```
>node --version
>npm --version
```

run Node.js

Create .js file

Then run the Node.js file

```
>node [file]
```

```
# Run a JavaScript file
>node app.js

# Run with additional arguments
>node app.js arg1 arg2

# Run in watch mode
# (restarts on file changes)
>node --watch app.js
```

```
// args.js
console.log('All arguments:', process.argv);
console.log('First argument:', process.argv[2]);
console.log('Second argument:', process.argv[3]);
```

Run with Arguments

```
> node args.js hello world

// Output:
All arguments: ['/path/to/node',
'/path/to/args.js', 'hello', 'world']
First argument: hello
Second argument: world
```

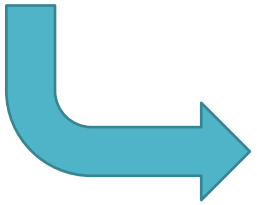
npm

npm is the default **package manager** for the Node.js JavaScript runtime environment

npm used to install and manage third-party packages (libraries)

```
# Common npm commands
npm init # Initialize a new project
npm install # Install dependencies
npm update # Update packages
npm audit # Check for vulnerabilities
```

```
// installs Express web framework
>npm install express
```



```
const express = require('express');
const app = express();
app.get('/', (req, res) => res.send('Hello World!'));
app.listen(8080);
```

built-in modules

built-in modules help you work with files, servers, paths, the operating system, and more

importing built-in modules with require()

```
const os = require('os');
console.log(os.platform());
```

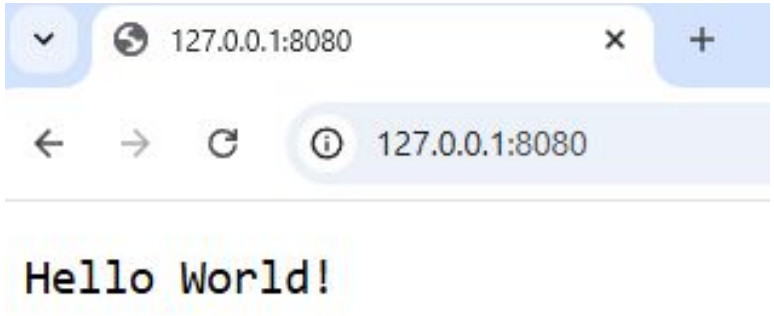


```
win32
```

Simple Web Server

```
const http = require('http');

http.createServer((req, res) => {
  res.writeHead(200, {'Content-Type': 'text/plain'});
  res.end('Hello World!');
}).listen(8080);
```




Asynchronous

Asynchronous Programming

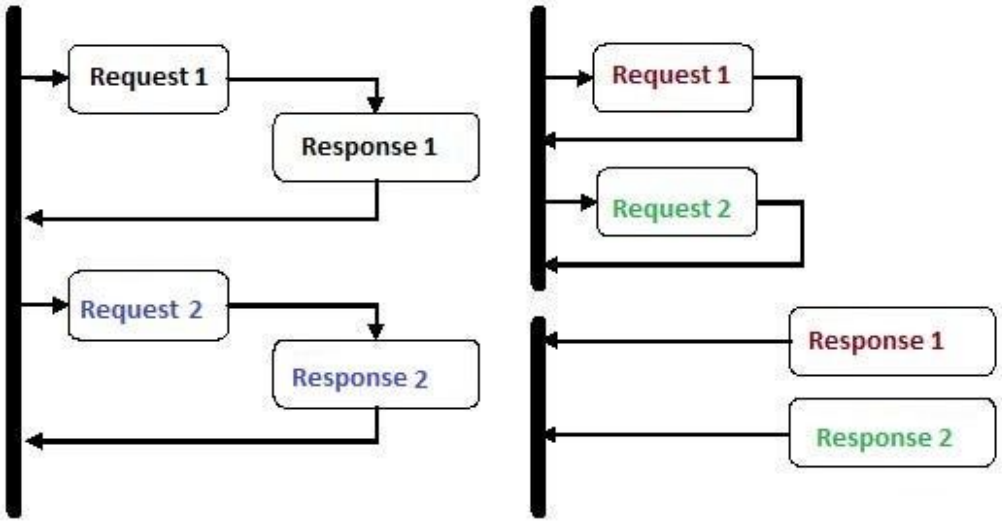
asynchronous operations let program do other work while waiting for tasks like file I/O or network requests to complete

Asynchronous code lets Node.js handle many requests at once, without waiting for slow operations like file or database access.

This makes Node.js great for servers and real-time apps.

Synchronous

- Blocks execution until complete
- Simple to understand
- Can cause delays
- Uses functions like `readFileSync`



Asynchronous

- Non-blocking execution
- Better performance
- More complex to handle
- Uses callbacks, promises, or async/await

Synchronous File

```
const fs = require('fs');

console.log('1. Starting sync read...');
const data = fs.readFileSync('myfile.txt', 'utf8');
console.log('2. File contents:', data);
console.log('3. Done reading file');
```

Output will be in order:

1→ 2 → 3

Asynchronous File

```
const fs = require('fs');

console.log('1. Starting async read...');
fs.readFile('myfile.txt', 'utf8', (err, data) => {
  if (err) throw err;
  console.log('2. File contents:', data);
});

console.log('3. Done starting read operation');
```

Output will be in order:

1 → 3 → 2

Callback

A **callback function** is a function passed into another function as an argument, which is then invoked inside the outer function to complete some kind of routine or action

```
function mainFunction(callback) {
    // Perform some operations

    // Execute the callback function
    callback();
}

function callbackFunction() {
    // console.log("Callback function executed!");
}

// Call the main function,
// passing the callback function as an argument
mainFunction(callbackFunction);
```

```
function calSum(num1, num2) {
    console.log("Calculating...");
    setTimeout(() => {
        console.log("Calculate completed");
        return num1 + num2;
    }, 2000);
}

function displayVal(val) {
    console.log("output = "+val);
}

let out = calSum(3, 5);
displayVal(out);
```

```
Calculating...
output = undefined
Calculate
completde
```

callback function

```
function calSum(num1, num2, callback) {
    console.log("Calculating...");
    setTimeout(() => {
        console.log("Calculate completed");
        let sum = num1 + num2;
        callback(sum);
    }, 2000);
}

function displayVal(val) {
    console.log("output = " + val);
}

calSum(3,5,displayVal);
```

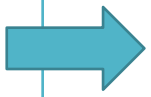
```
Calculating...
Calculate completed
output = 8
```

`displayVal` is a called a **callback function**.
It is passed to `calSum()` as an **argument**.

Callback with arrow function

```
function calSum(num1, num2, callback) {
    console.log("Calculating...");
    setTimeout(() => {
        console.log("Calculate completed");
        let sum = num1 + num2;
        callback(sum);
    }, 2000);
}


calSum(3,5, (val) => {
    console.log("output = " + val);
});
```



Calculating...
Calculate completed
output = 8



```
calSum(3,5, function(val) {
    console.log("output = " + val);
});
```

```
function downloading(filename, callback) {
    console.log("Download start ......" + filename);
    setTimeout(() => {
        callback(filename);
    }, 2000);
}


function completed(filename) {
    console.log("Download completed .." + filename);
 downloading("file1", completed);
```



```
Download start .....file1
Download completed ..file1
```

```
 downloading("file1", completed);
 downloading("file2", completed);
 downloading("file3", completed);
```



```
Download start .....file1
Download start .....file2
Download start .....file3
Download completed ..file1
Download completed ..file2
Download completed ..file3
```




```
function downloading(filename, callback) {
    console.log("Download start ......" + filename);
    setTimeout(() => {
        callback(filename);
    }, 2000);
}
function completed(filename) {
    console.log("Download completed .." + filename);


downloading("file1", (filename) => {
    console.log("Download completed .." + filename);
    downloading("file2", (filename) => {
        console.log("Download completed .." + filename);
        downloading("file3", completed)
```



```
Download start .....file1
Download completed ..file1
Download start .....file2
Download completed ..file2
Download start .....file3
Download completed ..file3
```

Callback hell



Promise

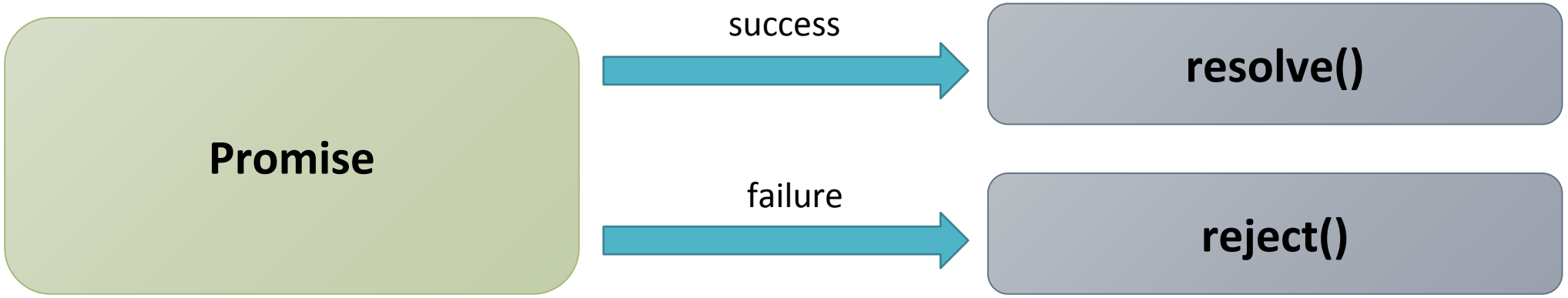
A Promise allows you to execute a series of asynchronous operations in sequence

Allows multiple asynchronous operations to run in sequence

Reduces callback hell by eliminating deeply nested functions.

Each **then()** returns a new promise, allowing further chaining.

Error handling is easier with a single .**catch()** for the entire chain.

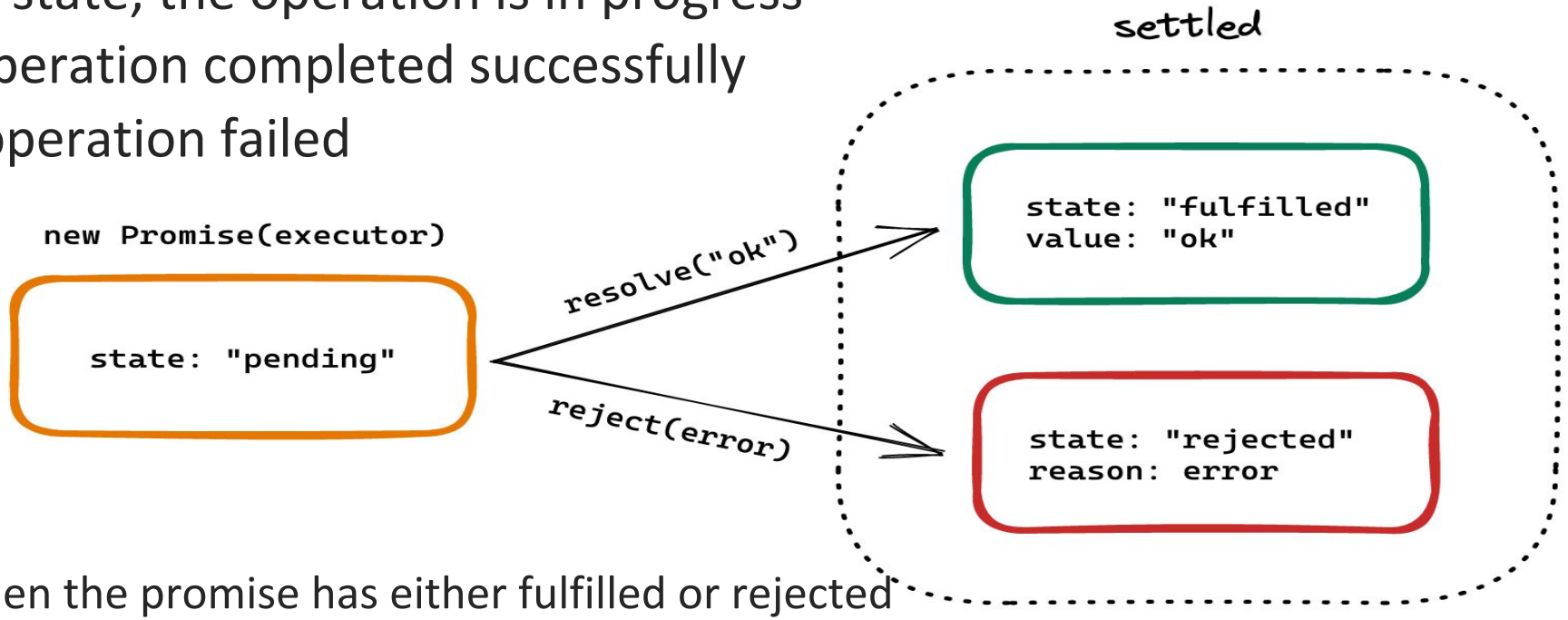


Promise States

myPromise.state	myPromise.result
"pending"	undefined
"fulfilled"	a result value
"rejected"	an error object

a promise can be in one of three states

- **Pending** — initial state, the operation is in progress
- **Fulfilled** — the operation completed successfully
- **Rejected** — the operation failed



A promise is settled when the promise has either fulfilled or rejected

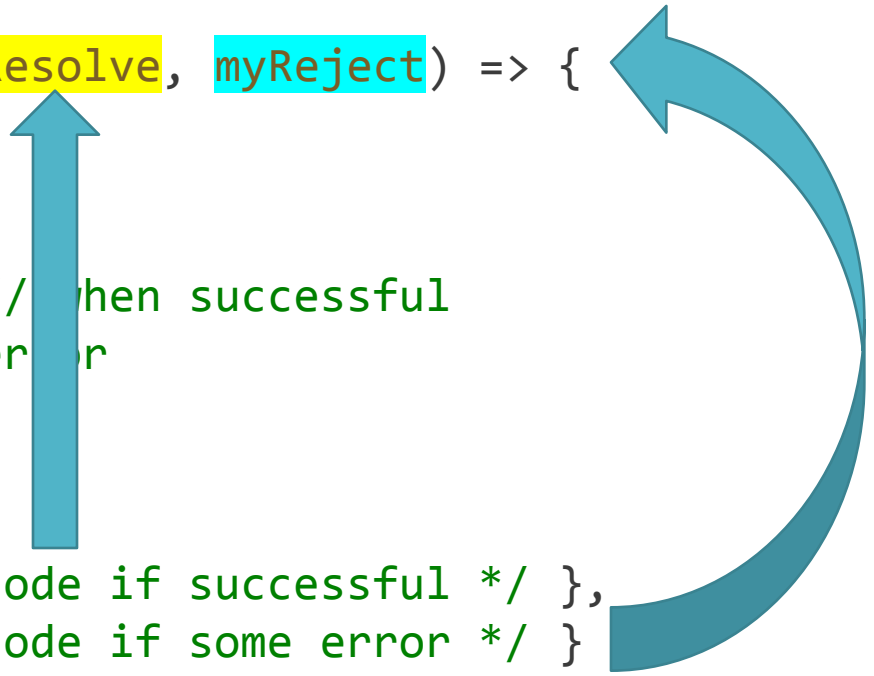
Once a promise is settled (either fulfilled or rejected), its state cannot change.

Promise syntax

```
const myPromise = new Promise((myResolve, myReject) => {
    let sucecss = true;
    // put code here

    if (sucecss) { myResolve(); }// when successful
    else { myReject(); } // when er r
});


myPromise.then(  (value) => { /* code if successful */ },
                 (error) => { /* code if some error */ }
);
```



Example :

promise

```
const downloadPromise = new Promise((myResolve, myReject) => {
    console.log("start download");
    setTimeout(() => {
        let success = Math.random() > 0.5;
        if (success) { myResolve(); }
        else { myReject(); }
    }, 2000);
})


const successMsg = () => { console.log("download complete"); }
const errorMsg = () => { console.log("download error") }


downloadPromise.then(successMsg, errorMsg);
```



```
downloadPromise.then(successMsg).catch(errorMsg);
```

Example : promise - function

```
function downloadPromise(filename) {
    return new Promise((myResolve, myReject) => {
        console.log("start download  " + filename);
        setTimeout(() => {
            let success = Math.random() > 0.5;
            if (success) { myResolve(filename); }
            else { myReject(filename); }
        }, 2000);
    })
}
const successMsg = (fn) => { console.log("download "+fn+" complete"); }
const errorMsg = (fn) => { console.log("download "+fn+" error") }
```

```
downloadPromise("file1")
    .then(successMsg)
    .catch(errorMsg);
```

start download file1
download file1 complete



start download file1
download file1 error

Example : promise - function

```
function downloadPromise(filename) {
    return new Promise((myResolve, myReject) => {
        console.log("start download ... " + filename);
        setTimeout(() => {
            let success = Math.random() > 0.5;
            if (success) {
                myResolve("download complete ..."+ filename);
            }
            else {
                myReject("download error ..."+ filename);
            }
        }, 2000);
    })
}

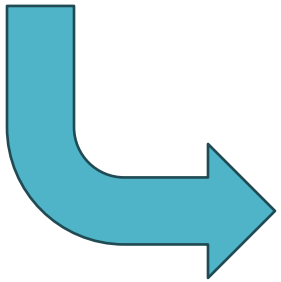
downloadPromise("file1")
    .then(result => { console.log(result); })
    .catch(err => { console.log(err) })
```

start download ... file1
download complete ...file1
End download

```
downloadPromise("file1")
    .then(result => { console.log(result); })
    .catch(err => { console.log(err) })

downloadPromise("file2")
    .then(result => { console.log(result); })
    .catch(err => { console.log(err) })

downloadPromise("file3")
    .then(result => { console.log(result); })
    .catch(err => { console.log(err) })
```



start download ... file1
start download ... file2
start download ... file3
download error ...file1
download complete ...file2
download error ...file3

Promise hell

```
downloadPromise("file1")
    .then(result => {
        console.log(result);
        downloadPromise("file2")
            .then(result => {
                console.log(result);
                downloadPromise("file3")
                    .then(result => { console.log(result); })
                    .catch(err => { console.log(err) })
            })
            .catch(err => { console.log(err) })
    })
    .catch(err => { console.log(err) })
```

start download ... file1
download error ...file1

start download ... file1
download complete ...file1
start download ... file2
download error ...file2

start download ... file1
download complete ...file1
start download ... file2
download complete ...file2
start download ... file3
download error ...file3

Promises chaining

Promise chaining in JavaScript allows for sequential execution of asynchronous operations, where the result of one operation is passed as input to the next.

```
somePromiseFunction() // Returns a Promise
    .then(function (result1) {
        // Handle result1 from the first promise
        // Perform another asynchronous operation or return a value
        return anotherPromiseFunction(result1); // Return a new Promise or a value
    })
    .then(function (result2) {
        // Handle result2 from the second promise (or the value returned from the previous .then)
        // Continue chaining or perform final actions
        return finalValue; // Return a value or another Promise
    })
    .catch(function (error) {
        // Handle any errors that occur in any part of the chain
        console.error("An error occurred:", error);
    })
    .finally(function () {
        // This block always executes, regardless of success or failure
```

Promises chaining

```
function downloadPromise(filename, prevData) {
    return new Promise((myResolve, myReject) => {
        console.log("start download ... " + filename);
        setTimeout(() => {
            if (Math.random() > 0.5) {
                console.log("download Complete")
                myResolve(prevData + " [data from " + filename + "] ");
            }
            else { myReject(filename); }
        }, 2000);
    })
}
downloadPromise("file1", "")
    .then(result => {
        console.log(`First result: ${result}`);
        return downloadPromise("file2", result);
    })
    .then(result => {
        console.log(`Second result: ${result}`);
        return downloadPromise("file3", result);
    })
    .then(result => {
```

```
start download ... file1
some error occur on file1
```

```
start download ... file1
download Complete
First result:  [data from file1]
start download ... file2
some error occur on file2
```

```
start download ... file1
download Complete
First result:  [data from file1]
start download ... file2
download Complete
Second result:  [data from file1]
[data from file2]
start download ... file3
some error occur on file3
```

https://javascript.info/promise-chaining

Promise method

Instance Methods

`then(onFulfilled, onRejected)`
Handles fulfillment or rejection

`catch(onRejected)`
Handles rejections

`finally(onFinally)`
Runs after promise settles

Static Methods

`Promise.all(iterable)`
Waits for all promises to resolve

`Promise.race(iterable)`
Waits for first promise to settle

`Promise.allSettled(iterable)`
Waits for all to settle

Utility Methods

`Promise.resolve(value)`
Creates a resolved promise

`Promise.reject(reason)`
Creates a rejected promise

Async/Await

Async/await is a modern way to handle asynchronous operations in Node.js, building on top of Promises

Async/await is basically Promises with a more readable syntax. This makes your code cleaner and more maintainable.

async function

await expression

async: Used to declare an asynchronous function that **returns a Promise**

```
async function fetchData() {
// ...
}
```

await: Used to pause execution until a Promise is resolved, can only be used inside async functions

```
let result = await somePromise;
```

Simple
Async/Await

```
function innerProcess() {
    return new Promise((resolve) => {
        setTimeout(() => {resolve('success');}, 2000);
    });
}

async function allProcess() {
    console.log('start');
    const result = await innerProcess();//return Promise
    console.log("end with " + result);
}

allProcess();
```

start
end with success

```
async function downloadfile(filename, prevData) {
    let promise = new downloadPromise(filename, prevData);
    console.log("start download ... " + filename);
    let result = await promise;
    console.log(`result: ${result}`);// do when success
    //return result;
}

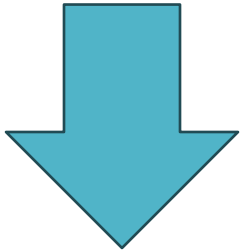
function downloadPromise(filename, prevData) {
    return new Promise((myResolve, myReject) => {
        setTimeout(() => {
            if (Math.random() > 0.5) {
                console.log("<download Complete>");
                myResolve(prevData + " [data from " + filename + "] ");
            } else {
                console.log("<download Error>");
                myReject(filename);
            }
        }, 2000);
    });
}
```

start download ... file1
<download Complete>
result: [data from file1]

start download ... file1
<download Error>
some error occur on file1

```
downloadfile("file1", "")
    .catch((fn) => { console.log("some error occur on " + fn) });
downloadfile("file2", "")
    .catch((fn) => { console.log("some error occur on " + fn) });
downloadfile("file3", "")
    .catch((fn) => { console.log("some error occur on " + fn) });
```

start download ... file1
start download ... file2
start download ... file3
<download Complete>
result: [data from file1]
<download Error>
some error occur on file2
<download Complete>
result: [data from file3]



```
downloadfile("file1", "")
    .then((result) => downloadfile("file2", result))
    .then((result) => downloadfile("file3", result))
    .catch((fn) => {
        console.log("some error occur on " + fn)
     });
```

start download ... file1
<download Complete>
result: [data from file1]
start download ... file2
<download Complete>
result: [data from file1] [data from file2]
start download ... file3
<download Error>
some error occur on file3

```
async function downloadAll(filename1, filename2, filename3) {
    let result;
    console.log("start download ... " + filename1);
    result = await new downloadPromise(filename1, "");
    console.log(`result: ${result}`);

    console.log("start download ... " + filename2);
    result = await new downloadPromise(filename2, result);
    console.log(`result: ${result}`);


    console.log("start download ... " + filename2);
    result = await new downloadPromise(filename3, result);
    console.log(`result: ${result}`);
}


downloadAll("file1", "file2", "file3")
   .catch((fn) => { console.log("some error occur on " + fn) });
```

```
start download ... file1
<download Complete>
result:  [data from file1]
start download ... file2
<download Complete>
result:  [data from file1]
[data from file2]
start download ... file2
<download Complete>
result:  [data from file1]
[data from file2]  [data
from file3]
```

```
start download ... file1
<download Complete>
result:  [data from file1]
start download ... file2
<download Error>
some error occur on file2
```



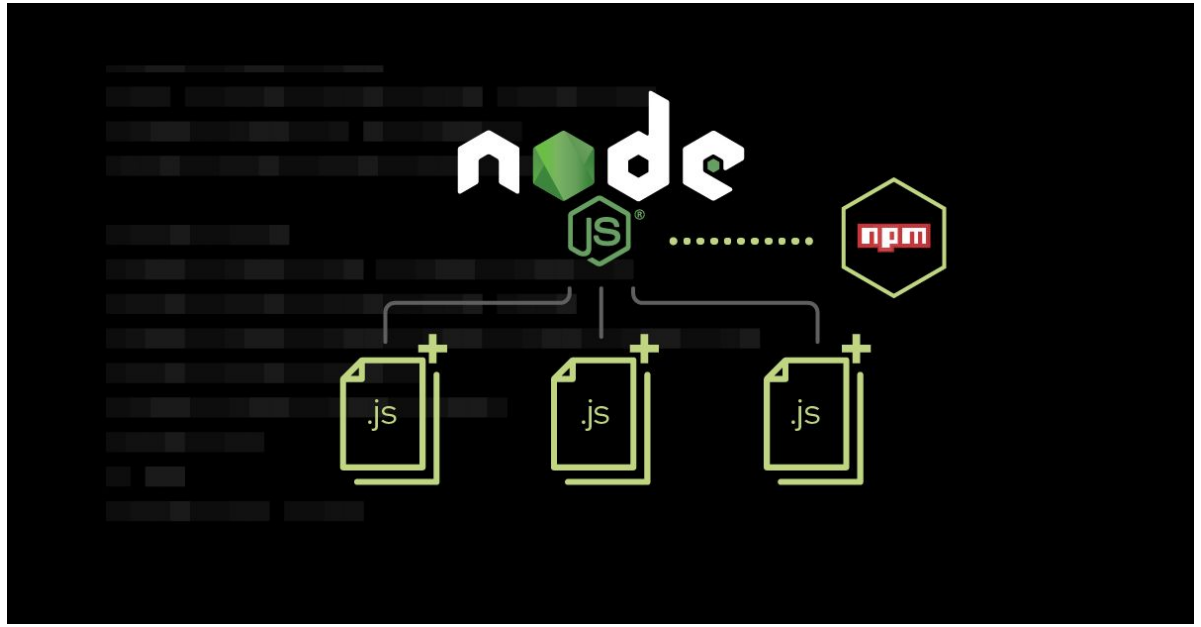
Module

Node.js Modules

Modules are the building blocks of Node.js applications, allowing you to organize code into logical, reusable components.

Benefits of Using Modules

- *Organizing code into manageable files*
- *Encapsulating functionality*
- *Preventing global namespace pollution*
- *Improving code maintainability and reusability*



Module Systems

module system enables you to organize your code into reusable pieces of functionality

COMMONJS

CommonJS is traditional module system

File extension .js

synchronous module system

- execution pauses until the required module is fully loaded

defined using the require() function

ES MODULE (ESM)

ECMAScript module is the newer module introduce in ECMAScript 6 (ES6)

File extension .mjs

asynchronous module system

- Supports both synchronous and asynchronous loading, gives better performance in web applications

use an import statement to load modules

CommonJS

```
// math.js
module.exports = {
  add: function(a, b) {
    return a + b;
  },
  subtract: function(a, b) {
    return a - b;
  }
}
```

```
// app.js
const math = require('./math');

console.log(math.add(2, 3));
// Output: 5

console.log(math.subtract(5, 3));
// Output: 2
```

ES module

```
// math.js

export function add(a, b) {
  return a + b;
}

export function subtract(a, b) {
  return a - b;
}
```

```
// app.js
import * as math  from './math.js';

console.log(math.add(2, 3));
// Output: 5

console.log(math.subtract(5, 3));
// Output: 2
```

Note : Node.js supports both CommonJS and ES but not all browsers support ES modules

Module types

Node.js modules can be categorized into three main types

Custom Modules:

The modules created by developers within their own projects to organize and reuse code.

Core Built-in Modules:

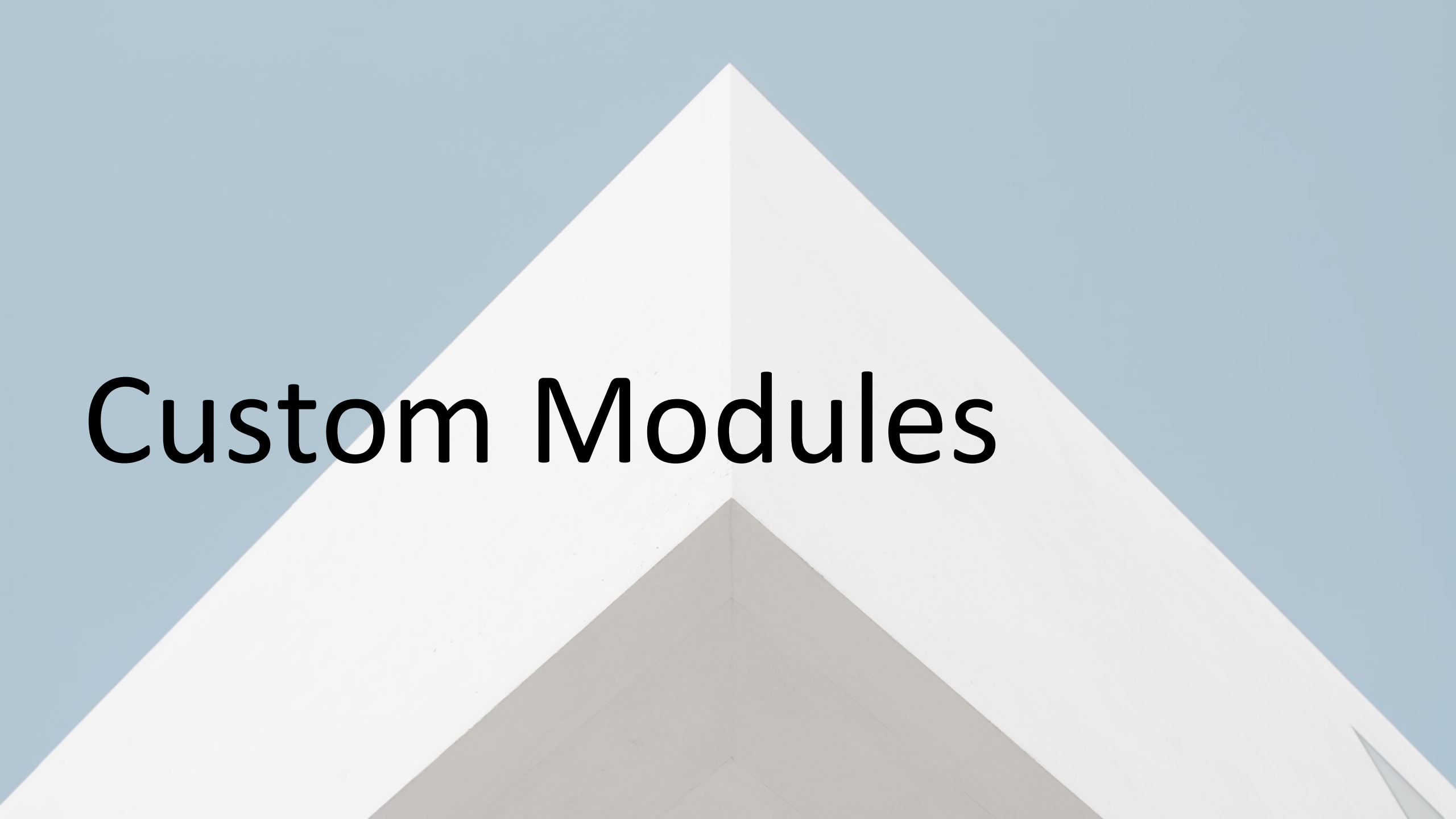
The built-in modules provided by NodeJS.

They provide essential functionalities for common tasks and do not require separate installation.

Third-Party Modules:

The external libraries created and maintained by other developers and shared through platforms like npm.

Custom Modules



Custom Modules

Node.js treats each file in a Node project as a module that can export values and functions from the file.

module is a global object with an exports property

Use require() to import and module.exports to export

export

```
module.exports = { exportObj }
// or
exports.exportName = exportObj
```

import

```
const { exportName } = require(<filename>)
```

```
console.log(module)
```

```
{
  id: '<repl>',
  path: '.',
  exports: {},
  filename: null,
  loaded: false,
  children: [],
  paths: [
    'D:\\workspace\\vscode\\Nodejs\\repl\\node_modules',
    'D:\\workspace\\vscode\\Nodejs\\node_modules',
    'D:\\workspace\\vscode\\node_modules',
    'D:\\workspace\\node_modules',
    'D:\\node_modules',
    'C:\\Users\\user\\.node_modules',
    'C:\\Users\\user\\.node_libraries',
    'C:\\Program Files\\nodejs\\lib\\node'
  ]
}
```

myModule.js

```
//variable
const PI = 3.14159;
//function
const add = (a, b) => a + b;
const subtract = (a, b) => a - b;
//class
class MyClass {
    constructor(name) {
        this.name = name;
    }
    greet() {
        return `Hello, ${this.name}!`;
    }
}


// export an object with multiple properties
module.exports = {
    add,
    subtract,
```

Export module

```
//Exporting multiple items

//variable
exports.PI = 3.14159;

//function
exports.add = (a, b) => a + b;
exports.subtract = (a, b) => a - b;

//class
exports.Person = class Person {
    constructor(name) {
        this.name = name;
    }
    greet() {
        return `Hello, ${this.name}!`;
```

```
exports.PI = PI;
exports.add = add;
exports.subtract = subtract;
exports.Person = Person;
```

```
const { PI, add, subtract, Person } = require('./myModule');

console.log(PI);
console.log(add(5, 3));
console.log(subtract(10, 4));
const person = new Person('John');
console.log(person.greet());
```

```
const { PI, add, subtract } = require('./myModule1');
console.log(PI);
console.log(add(5, 3));
console.log(subtract(10, 4));

const { Person } = require('./myModule1');
const person = new Person('John');
console.log(person.greet());
```

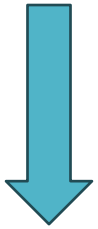
```
const myModule = require('./myModule');

console.log(myModule.PI);
console.log(myModule.add(5, 3));
console.log(myModule.subtract(10, 4));
const person = new myModule.Person('John');
console.log(person.greet());
```

```
3.14159
8
6
Hello, John!
```

Exporting a Single Item

```
module.exports = {
    add: function (x, y) {
        return x + y;
    },
    subtract: function (x, y) {
        return x - y;
    }
};
```

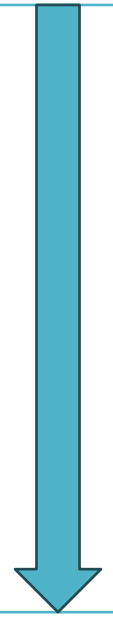


```
const cal = require('./myModule');

console.log(cal.add(3,5));
console.log(cal.subtract(3,5));
```

exports a single function

```
module.exports = (a, b) => a + b;
```



```
const mySum = require('./myModule1');

console.log(mySum(5, 3));
```

ES Module

ES Modules (ESM) is the official standard format for packaging JavaScript code for reuse

It was introduced in ES6 (ES2015) and is now supported in Node.js.

File Extension : .mjs (or .js with proper config)
Import Syntax : import
Export Syntax : export / export default
Import Timing : Static (parsed before execution)

ES Module - Export

Every module can have two different types of export

Module exports

Name export

Default export

Enabling ES Modules

Using the .mjs File Extension

Setting "type": "module" in package.json

```
{
  "name": "my-package",
  "version": "1.0.0",
  "type": "module"
}
```

Using the --input-type=module Flag

```
>node --input-type=module script.js
```

Named Exports

Export

```
// Multiple named exports
export function sayHello() {
  console.log('Hello');
}

export function sayGoodbye() {
  console.log('Goodbye');
}

// export list at the end
function add(a, b) {
  return a + b;
}

function subtract(a, b) {
  return a - b;
}

export { add, subtract };
```

Import

```
// Import specific named exports
import { sayHello, sayGoodbye } from './greetings.mjs';
sayHello(); // Hello
```

```
// Rename imports to avoid naming conflicts
import { add as sum, subtract as minus } from './math.mjs';
console.log(sum(5, 3)); // 8
```

```
// Import all named exports as an object
import * as math from './math.mjs';
console.log(math.add(7, 4)); // 11
```

Default Exports

You can have multiple named exports per module but only one default export.

```
// Only one default export per module
export default function() {
  console.log('Hi, How are you');
}
```

```
// Import default exports
import sayHi  from’./sayHi.mjs’;

sayHi();
```

```
// export default with a named function
function hi() {
   console.log('Hi, How are you');
}

export default hi;
```

```
// Import default exports
import sayHi  from’./sayHi.mjs’;

sayHi();
```

Mixed Exports

export

./myModule.mjs

```
const one = 1;
const two = 2;
const three = 3;

// named exports
export {one, two, three};

// default exports
export default three;

// named exports
export const four = 4;


// name and default export
export {one, two, three as default};
export const four = 4;
```

import

```
import { one, two, three, four }  from'./myModule.mjs';

console.log(one);
console.log(two);
console.log(three);
console.log(four);
```

```
1
2
3
4
```

```
// You can name the default import
import value from'./myModule.mjs';

console.log(value);
```

```
3
```

```
// Import both default and named exports
import III, { one,two }  from'./myModule.mjs';

console.log(one);
console.log(two);
console.log(III);
```

```
1
2
3
```

Module - Best Practices

MODULE ORGANIZATION

- Keep modules focused on **a single responsibility**
- Use meaningful file and directory names
- Group related functionality together
- Use index.js for module entry points

EXPORT PATTERNS

- Prefer named exports for utilities
- Use default exports for single-class modules
- Document your module's API
- Handle module initialization if needed

Core Build-in Module

- File system module
- HTTP module
- Path module
- URL module

Core Built-in Modules

Node.js provides several built-in modules that are compiled into the binary.

Here are some of the commonly used ones:

Core Modules	Description
fs	File system operations
http	HTTP server and client
path	File path utilities
os	Operating system utilities
events	Event handling
util	Utility functions
stream	Handle streaming data
url	URL parsing
querystring	URL query string handling
crypto	cryptographic operations (encryption, decryption)

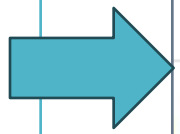
Core Built-in Modules

To use any built-in module, use the require() function:

```
const fs   = require('fs');
const path= require('path');
const http= require('http');
const util= require('util');
```

```
const http = require('http');

http.createServer((req, res) => {
  res.writeHead(200, {'Content-Type': 'text/plain'});
  res.end('Hello World!');
}).listen(8080);
```







File system module

File System Module

The fs (File System) module provides an API for interacting with the file system such as reading, writing, updating, and deleting files and directories

All file system operations can have synchronous and asynchronous forms depending upon user requirements

To use this File System module, use the require() method



Read Files
Write Files
Append Files
Close Files
Delete Files

Common Use Cases

File Operations

- Read and write files
- Create and delete files
- Append to files
- Rename and move files
- Change file permissions

Directory Operations

- Create and remove directories
- List directory contents
- Watch for file changes
- Get file/directory stats
- Check file existence

Advanced Features

- File streams
- File descriptors
- Symbolic links
- File watching
- Working with file permissions

Reading files: fs.readFile(), fs.readFileSync(), fs.createReadStream()

Writing files: fs.writeFile(), fs.writeFileSync(), fs.createWriteStream()

Managing directories: fs.mkdir(), fs.readdir(), fs.rmdir()

Checking file existence and permissions: fs.access(), fs.stat()

Read File

fs.readFile(path, options, callback);

```
const fs = require('fs');
fs.readFile("input.txt", "utf8", (err, data) => {
    if (err) {
        console.error("read File error : ", err);
        return;
    }
    console.log(data);
});
```

fs.readFile(): Asynchronously reads files, allowing other operations to continue while the file is being read.

Asynchronous

```
<< End Program >>
Hello How are you
This is the sample message
```

fs.readFileSync(): Synchronously reads files, blocking the event loop until the operation completes.

Synchronous

fs.readFileSync(path, options);

```
const fs = require('fs');
try {
    const data = fs.readFileSync("input.txt", "utf8");
    console.log(data);
}
catch (err) {
    console.error("read File error : ", err);
}
```

```
Hello How are you
This is the sample message
<< End Program >>
```

```
let data = fs.readFileSync("input.txt","utf8");
console.log(data);
```

```
Hello How are you
This is the sample messag
```

Read Binary file

```
let data = fs.readFileSync("input.txt");
console.log(data);
```

```
<Buffer 48 65 6c 6c 6f 20 48 6f 77 20 61
72 65 20 79 6f 75 0d 0a 54 68 69 73 20 69
73 20 74 68 65 20 73 61 6d 70 6c 65 20 6d
65 73 73 61 67 65>
```

```
// For binary data (like images), omit the encoding
let imgBin = fs.readFileSync("image.png");
console.log(imgBin.length, 'bytes');
```

120498 bytes

Write File

//create new or replace file

fs.writeFile(file, data, options, callback);

```
const fs = require("fs");

data = "Message to everyone";
fs.writeFile('output.txt', data, 'utf8', (err) => {
    if (err) {
        console.log(err)
        return
    }
    else console.log("Write file complet")
})
<< End Program >>
console.log("<< End Program >>");
Write file complete
```

Asynchronous

Synchronous

fs.writeFileSync(file, data, options);

```
const fs = require("fs");


data = "the message write to file";
try {
    fs.writeFileSync('output.txt', data, 'utf8');
    console.log("Write file complete");
} catch (err){
    console.log(err);
}
```

```
Write file complete
<< End Program >>
```

Append File

//create new or append file

fs.appendFile(file, data, options, callback);

```
const fs = require('fs');

data = "Message to everyone";
fs.appendFile('output.txt', data, 'utf8', (err) => {
    if (err) {
        console.log(err)
        return
    }
    else console.log("Append file complete")
})
console.log("<< End Program >>");
```

```
<< End Program >>
Append file complete
```

Asynchronous

fs.appendFileSync(file, data, options);

```
const fs = require('fs');

data = "the message write to file";
try {
    fs.appendFileSync('output.txt', data, 'utf8');
    console.log("Append file complete");
} catch (err){
    console.log(err);
}
```

Synchronous

```
Append file complete
<< End Program >>
```

Delete File

fs.unlink(file,

```
const fs = require("fs");

fs.unlink('output.txt', (err) => {
    if (err) {
        if (err.code === 'ENOENT')
          console.log('File does not exist');
        else
          console.error('Error :', err);
        return
    }
    else console.log("Remove file complete")
})
console.log("<< End Program >>");
```

Asynchronous

```
<< End Program >>
Remove file complete
```

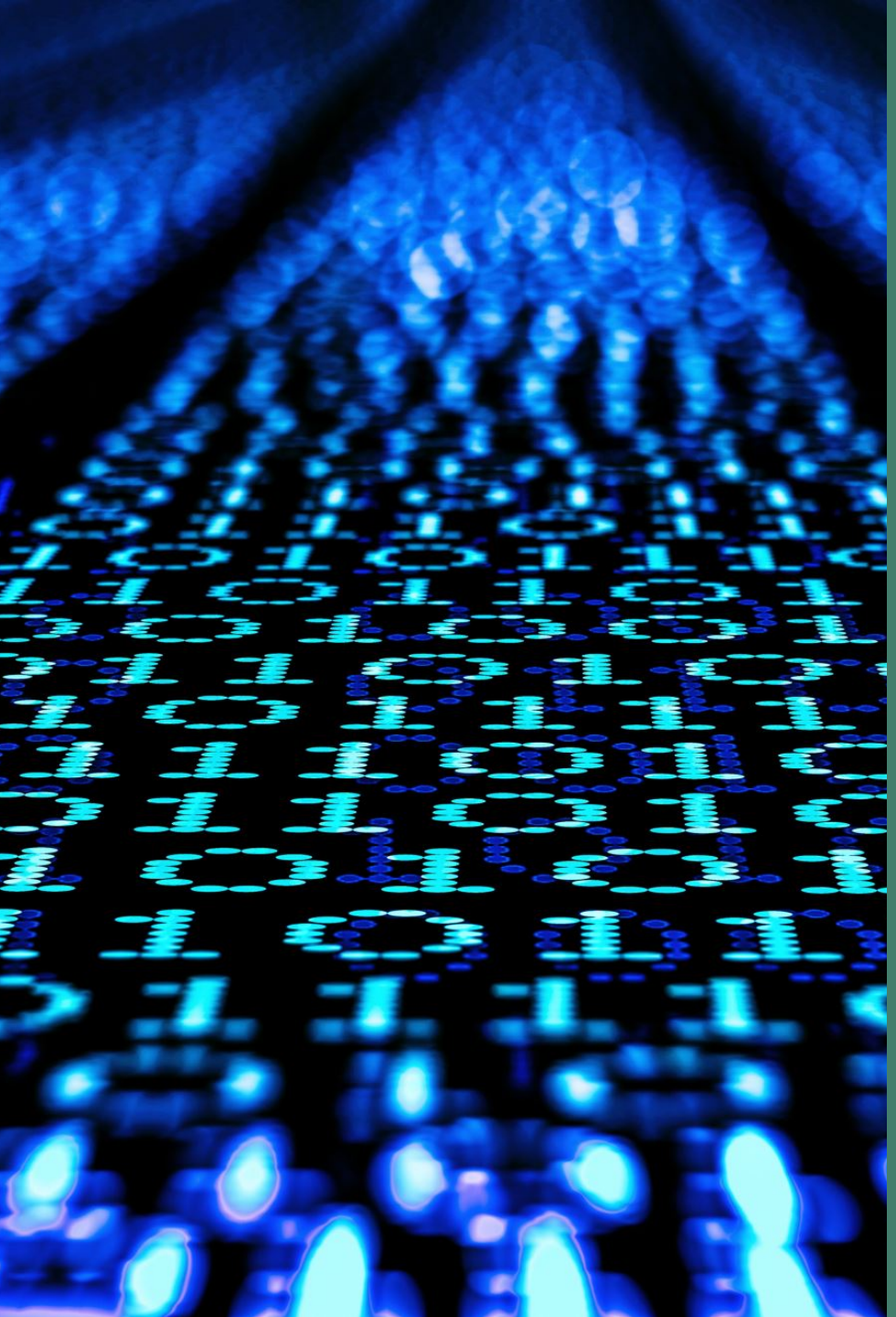
Synchronous

fs.unlinkSync(file)

```
var fs = require('fs');

try {
    fs.unlinkSync('output.txt');
    console.log("Remove file complete");
} catch (err){
    console.log(err);
}
console.log("<< End Program >>");
```

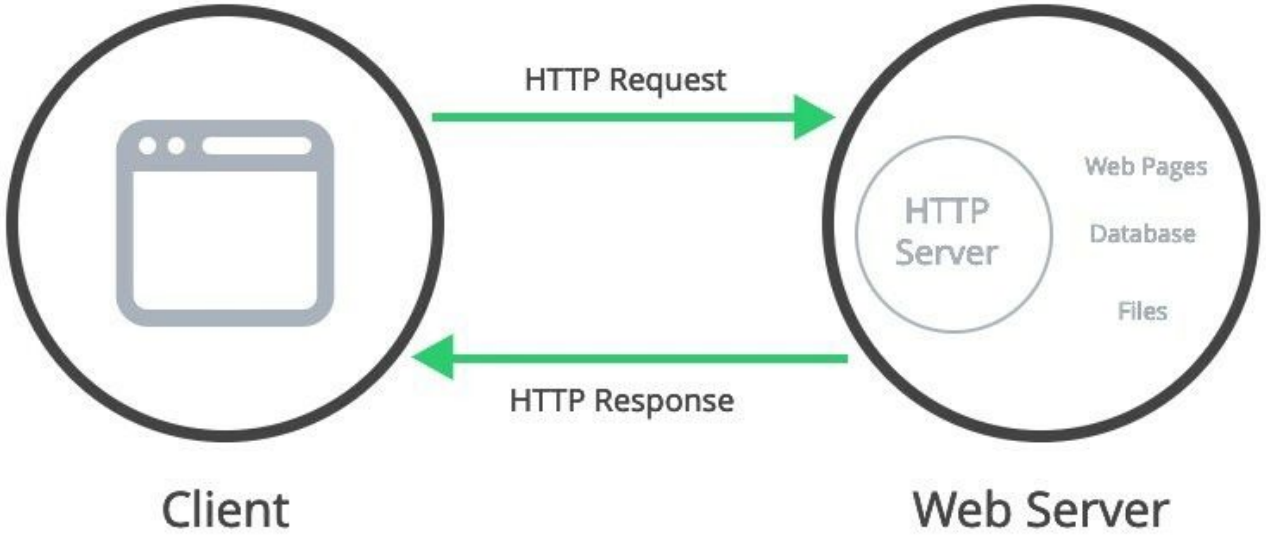
```
Remove file complete
<< End Program >>
```



HTTP module

HTTP Module

built-in HTTP module that enables to create HTTP servers and make HTTP requests



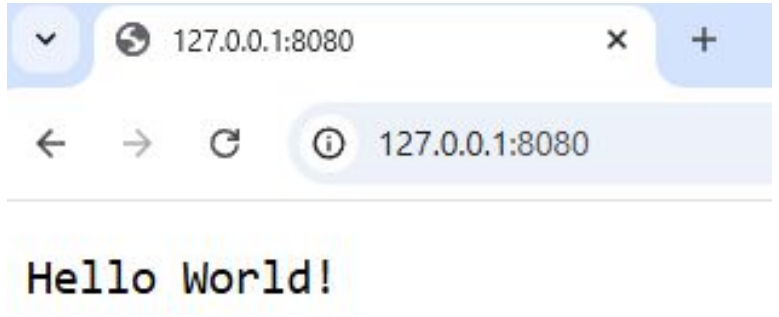
HTTP Module

createServer() method creates an HTTP server that listens for requests on a specified port and executes a callback function for each request.

Simple HTTP server

```
const http = require('http');

http.createServer((req, res) => {
  res.writeHead(200, {'Content-Type': 'text/plain'});
  res.end('Hello World!');
}).listen(8080);
```



Import HTTP module

Create server and define response

Start server

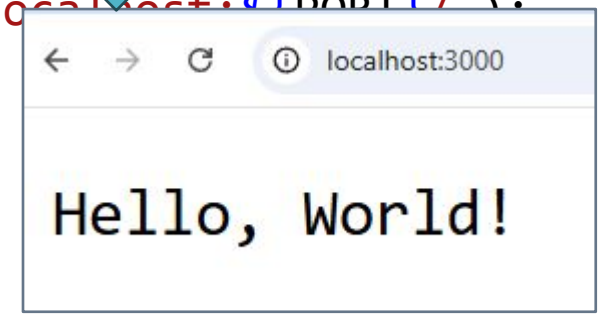
```
// Import the HTTP module
const http = require('http');

// Create a server object
const server = http.createServer((req, res) => {
  // Set the response HTTP header
  res.writeHead(200, { 'Content-Type': 'text/plain' });
  // Set the response body and Send
  res.end('Hello, World!\n');
});

// Define the port and start the server
PORT = 3000;
server.listen(PORT, 'localhost', () => {
  console.log(`Server running at http://localhost:${PORT}/`);
});
```

console

```
Server running at http://localhost:3000/
```



HTTP Headers

HTTP headers send additional information with your response

res.writeHead() method is used to set the status code and response headers

```
res.writeHead(200, {
  'Content-Type': 'text/html',
  'Cache-Control': 'no-cache, no-store, must-revalidate',
  'Set-Cookie': 'sessionid=abc123; HttpOnly'
});
```

Common Response Headers

Content-Type: Specifies the media type of the content (e.g., text/html, application/json)
Content-Length: The length of the response body in bytes
Location: Used in redirects (with 3xx status codes)
Set-Cookie: Sets HTTP cookies on the client
Cache-Control: Directives for caching mechanisms
Access-Control-Allow-Origin: For CORS support

HTTP Headers

Common HTTP Status Codes

Code	Message	Description
200	OK	Standard response for successful HTTP requests
201	Created	Request has been fulfilled and new resource created
301	Moved Permanently	Resource has been moved to a new URL
400	Bad Request	Server cannot process the request due to client error
401	Unauthorized	Authentication is required
403	Forbidden	Server refuses to authorize the request
404	Not Found	Requested resource could not be found
500	Internal Server Error	Unexpected condition was encountered

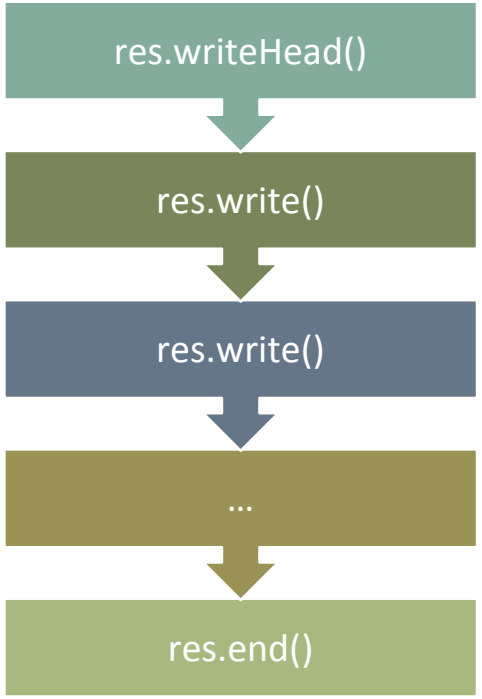
HTTP body

res.write(data , encoding)

sends a chunk of the response body to the client.

res.end(data, encoding)

sends the last chunk of data and signals the end of the response




```
http.createServer((req, res) => {
    res.writeHead(200, {'Content-Type':'text/html'});
    res.write('<h1>Hello world</h1>');
    res.write('<h3>Hello world</h3>');
    res.end('<p>How are you</p>');
}).listen(3000);
```

Hello world

Hello world

How are you

Request URLs

The req.url property contains the URL string that was requested, including any query parameters

```
http.createServer((req, res) => {
    console.log(req.url)
    console.log(req.method)

    res.writeHead(200, {'Content-Type':'text/plain'});
    res.end('Hello World!');


}).listen(3000);
```

http://localhost:3000

```
/
GET
```

http://localhost:3000/hello/how/are/you

```
/hello/how/are/you
GET
```

```
const http = require('http');
var fs = require('fs');


http.createServer((req, res) => {
    const reqPath = req.url;
    res.writeHead(200, { 'Content-Type': 'text/html' });
    if (reqPath == '/' || reqPath == '/home')
        res.end('<h1>Home</h1>’);

    else if (reqPath == '/product1')
        res.end('<h1>Product 1</h1>’);

    else if (reqPath == '/product2')
        res.end('<h1>Product 2</h1>’);

    else if (reqPath == '/product3')
        res.end('<h1>Product 3</h1>’);

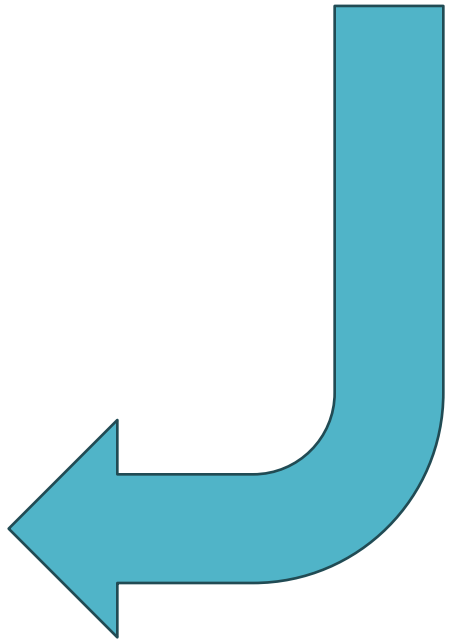
    else{
        res.end('<h1>Page not found</h1>’);
```

File Response

```
if (reqPath == '/' || reqPath === '/home')
    fs.readFile('page1.htm', function (err, data) {
        res.end(data);
    });
else if (reqPath == '/product1')
    fs.readFile('product1.htm', function (err, data) {
        res.end(data);
    });
else if (reqPath == '/product2')
    fs.readFile('product2.htm', function (err, data) {
        res.end(data);
    });
else if (reqPath == '/product3')
    fs.readFile('product3.htm', function (err, data) {
        res.end(data);
    });
else{
    res.end('<h1>Page not found</h1>’);
```

Response HTML with css file

```
<link rel="stylesheet" href="/style.css">
```



```
if (reqPath == '/' || reqPath === '/home')
    fs.readFile('page1.htm', function (err, data) {
        res.end(data);
    });
else if (reqPath == '/style.css')
    fs.readFile('style.css', function (err, data) {
        res.end(data);
    });
```



Path module

Path module

Path module is a built-in Node.js module that provides tools for handling and transforming file paths **across different operating systems**

Windows uses backslashes (\) and Linux and macOS use forward slashes (/)

__dirname provide the directory name of the current module

__filename provide the file name of the current module

path.join() Joins all given path segments together using the platform-specific separator

```
const path = require('path');

console.log(__dirname)
console.log(__filename)
console.log(path.join(__dirname,"image","product","food"))
console.log(path.join(__dirname,"/image","\product", "//food"))
console.log(path.join(__dirname,"../../image","product/store", "../food"))
```



```
d:\vscode\NodeHTTP
d:\vscode\NodeHTTP\pathTest.js
d:\vscode\NodeHTTP\image\product\food
d:\vscode\NodeHTTP\image\product\food
d:\image\product\food
```

File Response

```
if (reqPath == '/' || reqPath === '/home')
    fs.readFile(path.join(__dirname, "page1.htm"), function (err, data) {
        res.end(data);
    });
else if (reqPath == '/product1')
    fs.readFile(path.join(__dirname, "product1.htm"), function (err, data) {
        res.end(data);
    });
else if (reqPath == '/product2')
    fs.readFile(path.join(__dirname, "product2.htm"), function (err, data) {
        res.end(data);
    });
else if (reqPath == '/product3')
    fs.readFile(path.join(__dirname, "product3.htm"), function (err, data) {
        res.end(data);
    });
else{
    res.end('<h1>Page not found</h1>’);
```

path.parse()

Returns an object whose properties represent significant elements of the path

```
const path = require('path');

const pathInfo = path.parse('D:/users/docs/file.txt’);

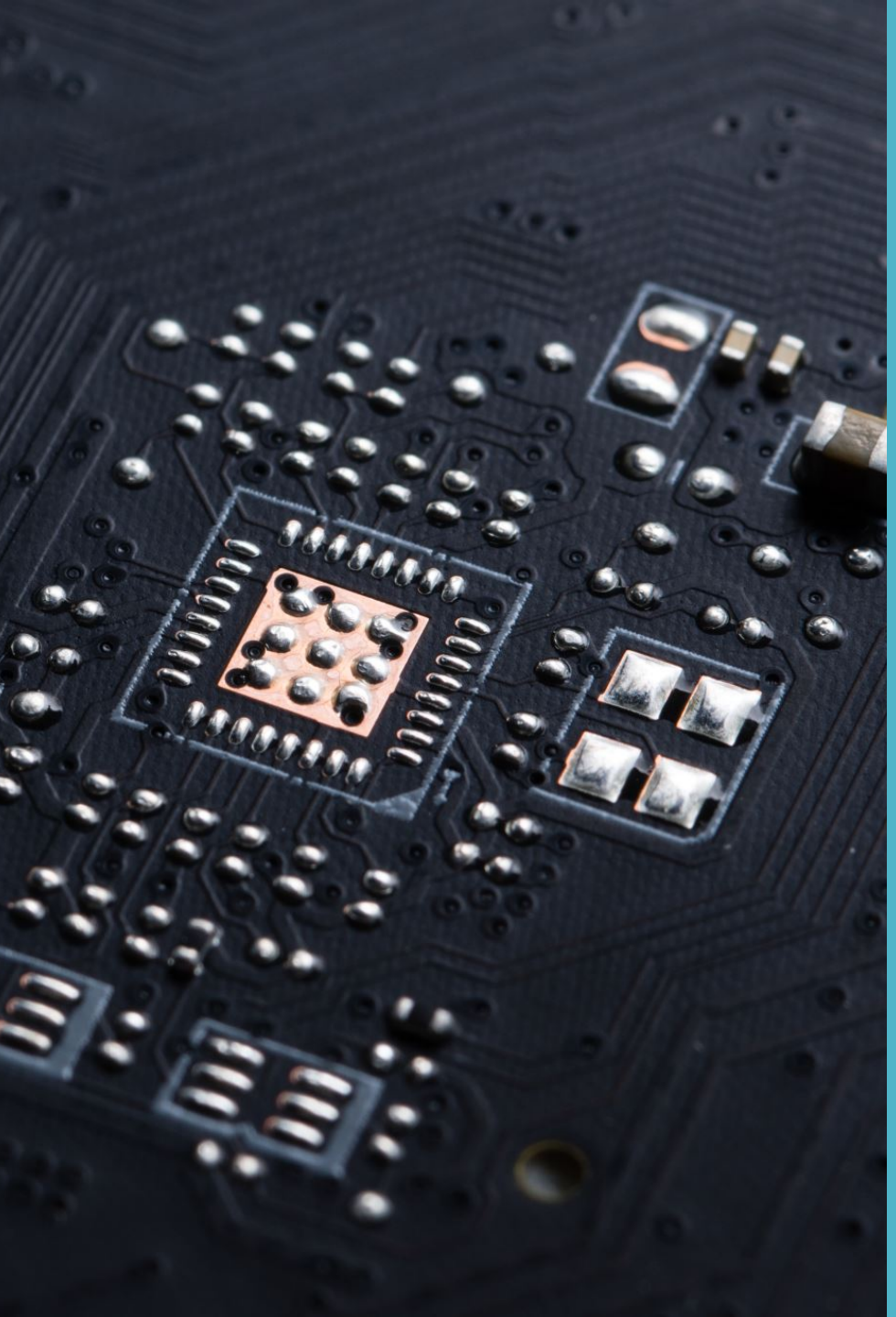
console.log(pathInfo);
```



```
{
  root: 'D:/',
  dir: 'D:/users/docs',
  base: 'file.txt',
  ext: '.txt',
  name: 'file’
}
```

```
console.log('root :', pathInfo.root)
console.log('directory :', pathInfo.dir)
console.log('file base :', pathInfo.base)
console.log('file name :', pathInfo.name)
console.log('file ext :', pathInfo.ext)
```

```
root : D:/
directory : D:/users/docs
file base : file.txt
file name : file
file ext : .txt
```



URL module

URL module

URL module provides utilities for URL resolution and parsing

url.parse() method, and it will return a URL object with each part of the address as properties

```
let url = require('url');

let adr = 'http://localhost:8080/product/food/default.htm?year=2017&month=february';

let q = url.parse(adr, true);

console.log("host",q.host);
console.log("pathname",q.pathname);
console.log("search",q.search);
```

```
host =  localhost:8080
pathname =  /product/food/default.htm
search =  ?year=2017&month=february
```

```
let url = require('url');
let adr = 'http://localhost:8080/product/food/default.htm?year=2017&month=february';
let q = url.parse(adr, true);
console.log(q)
```



```
Url {
  protocol: 'http:',
  slashes: true,
  auth: null,
  host: 'localhost:8080',
  port: '8080',
  hostname: 'localhost',
  hash: null,
  search: '?year=2017&month=february',
  query: [Object: null prototype] { year: '2017', month: 'february' },
  pathname: '/product/food/default.htm',
  path: '/product/food/default.htm?year=2017&month=february',
  href: 'http://localhost:8080/product/food/default.htm?year=2017&month=february'
}
```

```
let qdata = q.query;
console.log(qdata.month);
console.log(q.query.year);
```



```
february
2017
```

```
const http = require('http');
var fs = require('fs');
let url = require('url');

http.createServer((req, res) => {
    const q = url.parse(req.url, true);
    const reqPath = q.pathname;

    res.writeHead(200, { 'Content-Type': 'text/html' });
    if (reqPath == '/' || reqPath == '/home')
        fs.readFile('page1.htm', function (err, data) {res.end(data);});

    else if (reqPath == '/product' && q.query.id == 1)
        fs.readFile('product1.htm', function (err, data) {res.end(data);});

    else if (reqPath == '/product' && q.query.id == 2)
        fs.readFile('product2.htm', function (err, data) {res.end(data);});

    else if (reqPath == '/product' && q.query.id == 3)
        fs.readFile('product3.htm', function (err, data) {res.end(data);});
```

URLSearchParams

provides utility methods to work with the query string of a URL

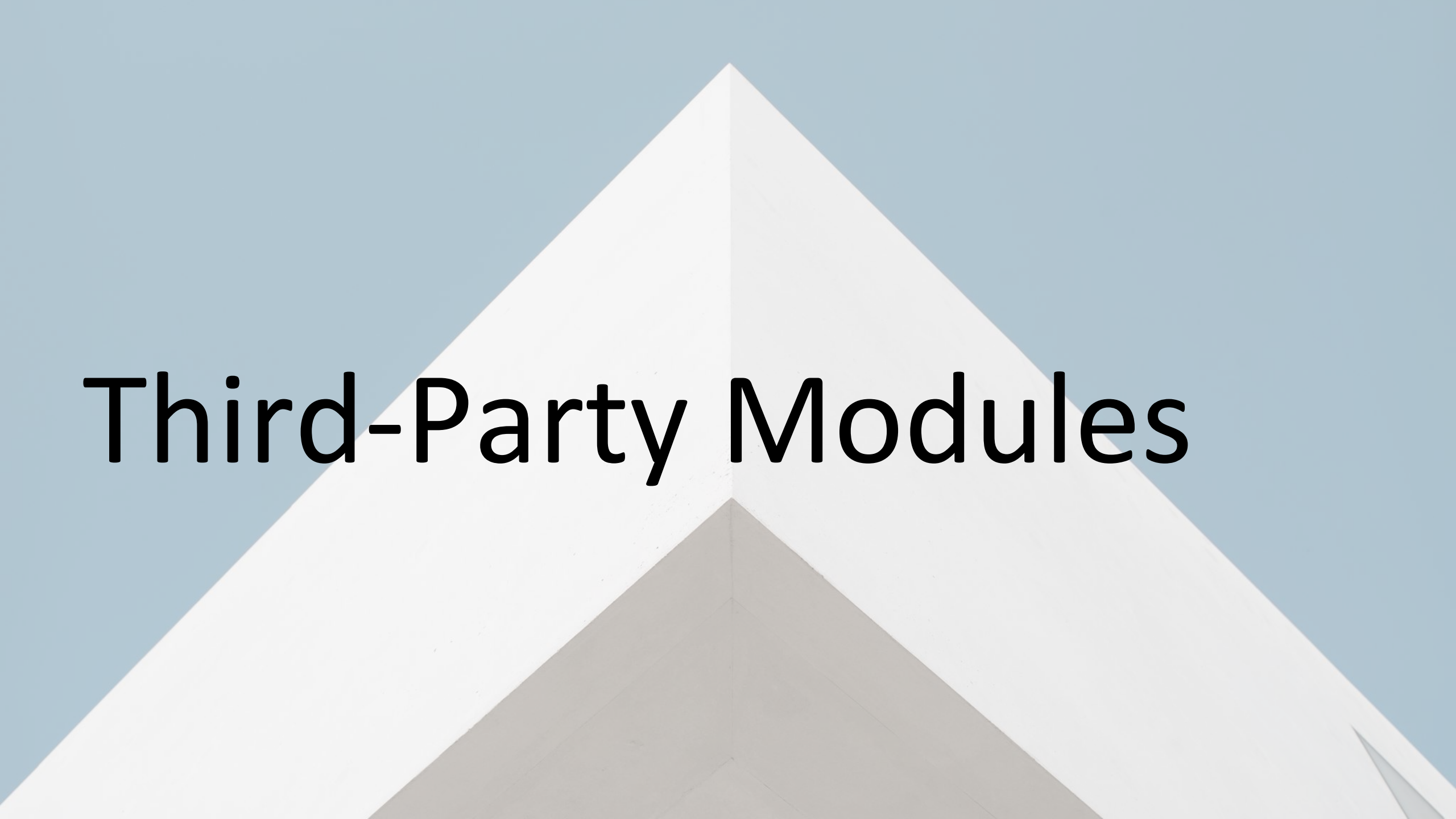
```
const { URL, URLSearchParams } = require('url');

const myURL = new URL('https://example.com/?name=John&lastName=Doe');

const params = new URLSearchParams(myURL.search);

// Get a parameter
console.log(params.get('name'));
// Add a parameter
params.append('age', 35);
// Delete a parameter
params.delete('lastName');
// Convert to string
console.log(params.toString());
```

```
John
name=John&age=35
```



Third-Party Modules

package.json

package.json is a special file that contains information about your app, such as its name, version, dependencies, scripts, and more.

This file is essential for managing and sharing Node.js projects, especially when using npm (Node Package Manager).

Creating package.json file

```
npm init
```

```
npm init -y
```



```
{
  "name": "my-node-app",
  "version": "1.0.0",
  "description": "A simple Node.js app",
  "main": "index.js",
  "scripts": {
    "start": "node index.js"
  },
  "author": "Your Name",
  "license": "ISC"
}
```

Adding Dependencies

```
npm install express
```

This command adds Express to your project and updates **package.json** automatically



```
"dependencies": {
  "express": "^5.1.0"
}
```

Working with package.json

Adding Dependencies

```
# Install and save to dependencies
npm install package-name

# Install and save to devDependencies
npm install --save-dev package-name

# Install exact version
npm install package-name@1.2.3
```

Updating Dependencies

```
# Update a specific package
npm update package-name

# Update all packages
npm update

# Check for outdated packages
npm outdated
```

Running Scripts

```
# Run a script
npm run script-name

# Run start script (can be called with just 'npm start')
npm start

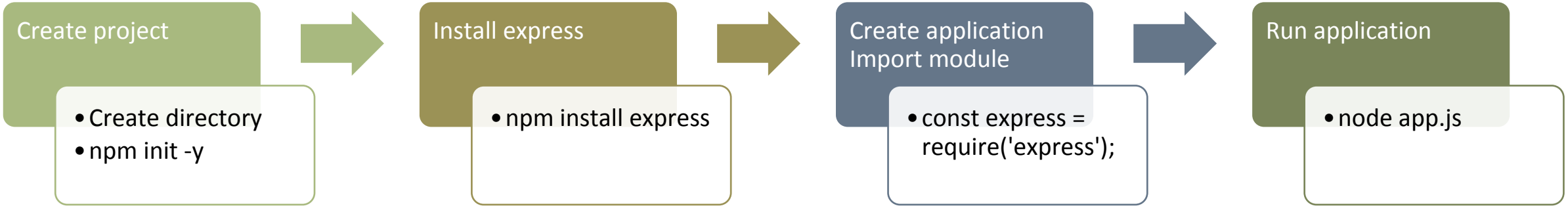
# Run test script (can be called with just 'npm test')
npm test
```


Express.js

Express.js

Express.js is the most popular **Node.js web application framework**, designed for building web applications and APIs

Getting Started



package.json

```
"dependencies": {
  "express": "^5.1.0"
}
```

Nodemon

Nodemon is a command-line tool that monitors the project directory and **automatically restarts** your node application when it detects any changes.

Install nodemon

```
npm install -g nodemon
```

Run application with nodemon

```
npx nodemon app.js
```

app.method(path, handler)

```
const express =require("express");

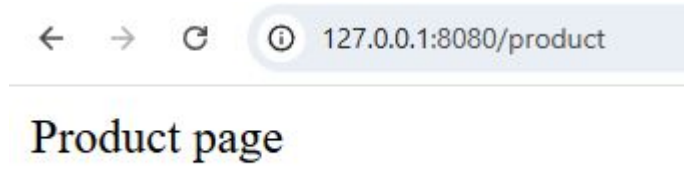

const app = express();
const port = 8080;


app.get('/', (req,res)=>{
    res.send("Home page");
})


app.get('/product', (req,res)=>{
    res.send("Product page");
})


app.listen(port,()=>{console.log("server start at port ",port)})
```





app.get – response for method GET

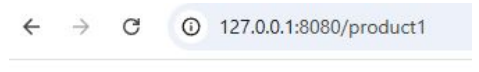
```
const express = require("express");
const path = require('path');

const app = express();
const port = 8080;

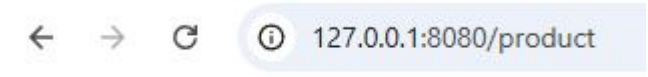
app.get(['/', '/home'], (req, res) => {
    res.sendFile(path.join(__dirname, "page1.htm"));
})
app.get('/product1', (req, res) => {
    res.sendFile(path.join(__dirname, "product1.htm"));
})
app.get('/product2', (req, res) => {
    res.sendFile(path.join(__dirname, "product2.htm"));
})
app.get('/product3', (req, res) => {
    res.sendFile(path.join(__dirname, "product3.htm"));
})
app.get('/{*any}', (req, res) => {
    res.status(404).send('PAGE NOT FOUND');
    //res.redirect('/');
```



Welcome



product1



PAGE NOT FOUND

Basic Routing

Routing refers to how an application responds to client requests to specific endpoints (URIs) using different HTTP methods (GET, POST, PUT, DELETE, etc.)

app.get() - Handle GET requests

app.post() - Handle POST requests

app.put() - Handle PUT requests

app.delete() - Handle DELETE requests

app.all() - Handle all HTTP methods

```
// Respond to GET request on the root route
app.get('/', (req, res) => { });

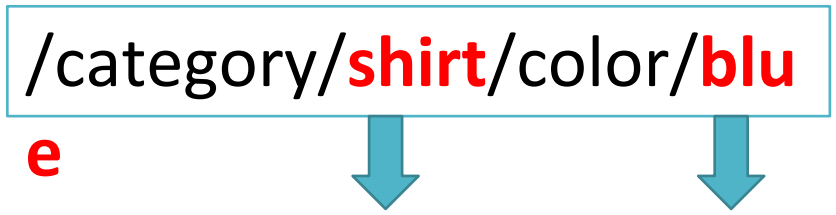
// Respond to GET request on the /about route
app.get('/about', (req, res) => { });

// Respond to POST request on the root route
app.post('/', (req, res) => { });

// Catch all other routes
app.all('*', (req, res) => { });
```

Route Parameters

Route parameters are named URL segments that capture values at specific positions in the URL




```
app.get('/category/:pType/color/:color', (req, res) => {
  res.send(`<p>category = ${req.params.pType}</p>
            <p>color = ${req.params.color}</p>`);
});
```

http://localhost:8080/category/**hat**/color/**gree**
n

category = hat

color = green

Query Parameters

Query parameters are key-value pairs that appear after the ? in a URL

They are automatically parsed by Express and available in **req.query**

/product?category=shirt&color=blue

```
app.get('/product', (req, res) => {
  res.send(`<p>category = ${req.query.pType}</p>
            <p>color = ${req.query.color}</p>`);
});
```

/product?pType=bag&color=pink

category = bag

color = pink

```
const { pType, color } = req.query;
console.log(pType, color);
```

Serving Static Files

Static files are unchanging resources that are delivered directly to a web browser without any server-side processing or modification during the request

EX. HTML, CSS, JavaScript, images, fonts

use express.static() to **access these files directly**

Set folder public to store static file

```
app.use(express.static(path.join(__dirname, 'public')));
```



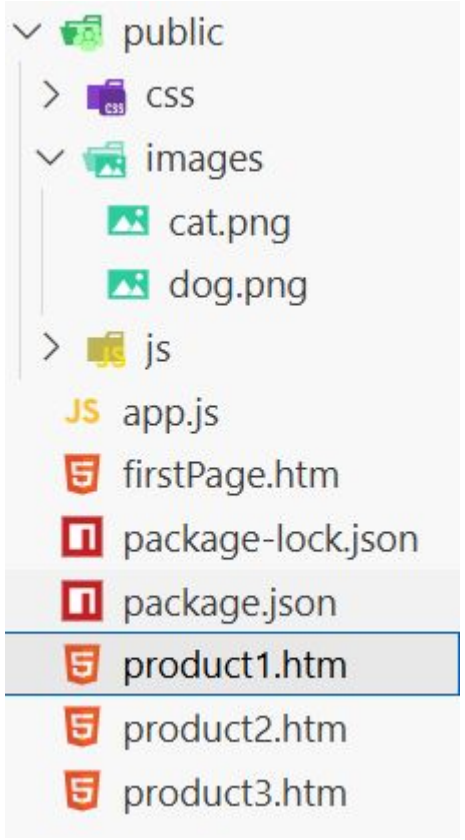
http://localhost:8080/cat.png



http://localhost:8080/dog.png








```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=<device-width>, initial-scale=1.0">
    <title>Document</title>
    <link rel="stylesheet" href="/css/style.css">
</head>
<body>
    <h1>Product1</h1>
    <p>This is product1 page</p>
    <img src="/images/dog.png" alt="" width="300px" >
</body>
</html>
```

Router module

For better organization, you can define routes in separate files using Express Router

Use the express.Router class to create modular, mountable route handlers

/routes/myRouter.j

```
s
const express = require('express');
const router = express.Router();
const path = require('path');
// Define routes
router.get(['/', '/home'], (req, res) => {
    res.sendFile(path.join(__dirname, "../firstPage.htm"));
})
router.get('/product1', (req, res) => {
    res.sendFile(path.join(__dirname, "../product1.htm"));
})
module.exports = router;
```

app.js

```
const express = require('express');
const path = require('path');
const myRouter = require(path.join(__dirname, './routes/myRouter'));

const app = express();
const port = 8080;

app.use(express.static(path.join(__dirname, 'public')));
app.use(myRouter);

app.listen(port, () => {
    console.log(`Server is running at http://localhost:${port}`);
});
```

For better organization, you can define routes in separate files using Express Router

/routes/user.j

```
const express = require('express');
const router = express.Router();

// Define routes
router.get('/', (req, res) => {
  res.send('Users home page');
});

router.get('/:id', (req, res) => {
  res.send(`User profile for ID:
${req.params.id}`);
});

module.exports = router;
```

/routes/product.j

```
const express = require('express');
const router = express.Router();

// Define routes
router.get('/', (req, res) => {
  res.send('Products list');
});

router.get('/:id', (req, res) => {
  res.send(`Product details for ID:
${req.params.id}`);
});

module.exports = router;
```

app.js

```
const express = require('express');
const path = require('path');

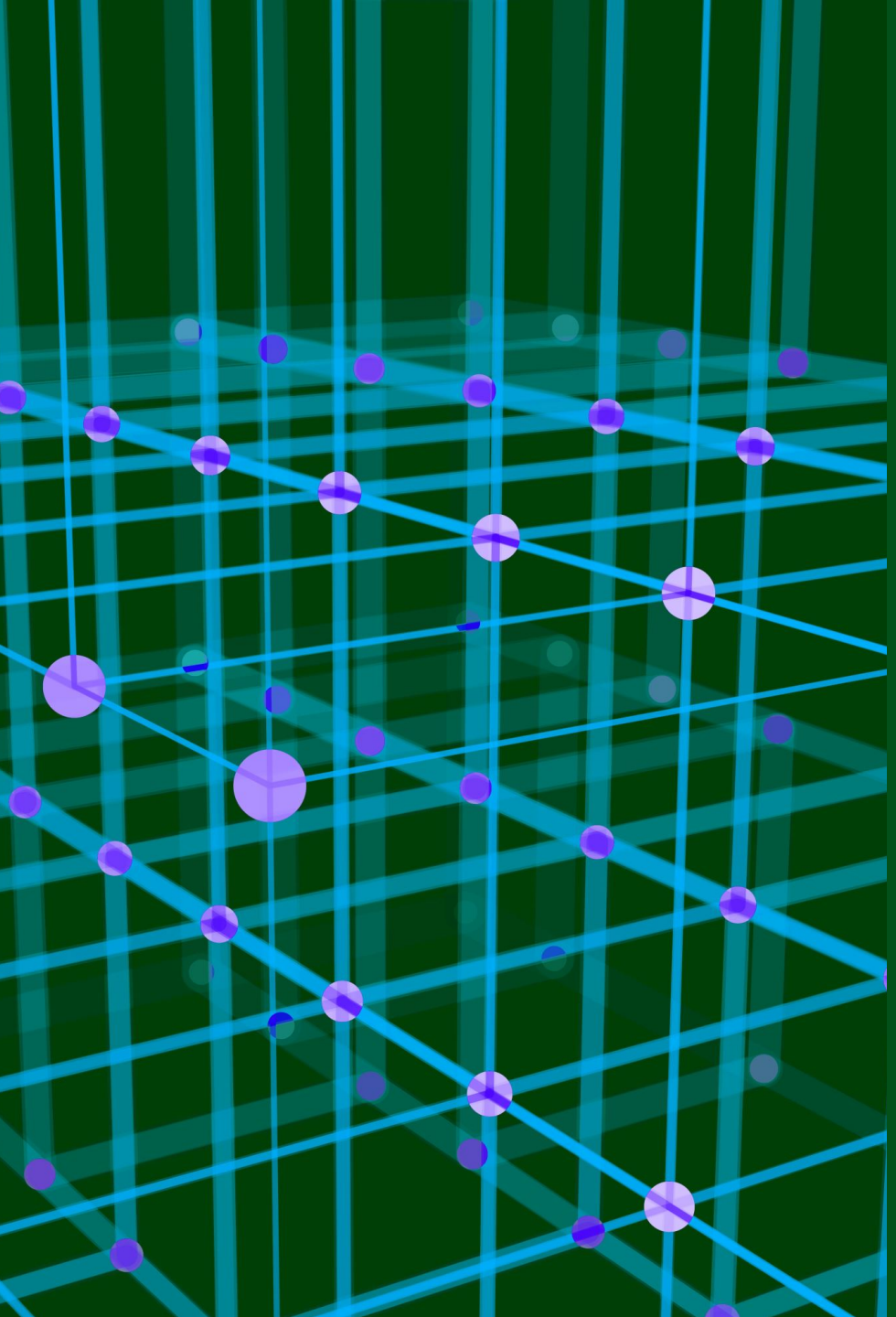
const myRouter = require(path.join(__dirname, './routes/myRouter'));
const usersRouter = require(path.join(__dirname, './routes/users'));
const productsRouter = require(path.join(__dirname, './routes/products'));

const app = express();
const port = 8080;

app.use(express.static(path.join(__dirname, 'public')));

app.use('/',myRouter);
app.use('/users', usersRouter);
app.use('/products', productsRouter);

app.listen(port, () => {
    console.log(`Server is running at http://localhost:${port}`);
});
```



Todo

Todo 1

1 เขียนโปรแกรมวัดดวง โดยการสุ่มตัวเลข 0-9 จำนวน 3 ครั้ง การสุ่มแต่ละครั้งจะต้องรอผลการสุ่มเป็นเวลา 2 วินาที โดยหากตัวเลขที่สุ่มได้ เป็นเลขคู่ จะได้รับการสุ่มครั้งถัดไป ผู้ชนะต้องสุ่มเลขแล้วเป็นเลขคู่ทั้ง 3 ครั้ง

เลือกใช้อย่างใดอย่างหนึ่ง

1.1 ใช้ callback

1.2 ใช้ promise

1.3 await / async

ตัวอย่างผลลัพธ์



```
Wait 2 second ...
Num 1 :  2

Wait 2 second ...
Num 2 :  6

Wait 2 second ...
Num 3 :  8

You win
```

```
Wait 2 second ...
Num 1 :  2

Wait 2 second ...
Num 2 :  5

You lost
```

```
Wait 2 second ...
Num 1 :  4

Wait 2 second ...
Num 2 :  6

Wait 2 second ...
Num 3 :  3

You lost
```

Todo 2

สร้างเว็บโปรโมทร้านอาหารจำนวน 3 หน้า **โดยใช้ Node.js** วางโครงสร้างหน้าเว็บดังนี้ (ใช้ไฟล์จากงาน HTML ที่เคยส่ง)

folder	file
/	index.htm
/info	menu.htm order.htm
/public/images	Image files
/public/css	CSS files

/home แสดง index.htm

/menu แสดง menu.htm

/order แสดง order.htm

/item/:name/price/:price
รับชื่อและราคา ผ่าน url (Route Parameters) เพื่อแสดงข้อมูล
โดยเพิ่ม link จากรายการอาหาร ที่แสดงในหน้า menu.htm ทุ

